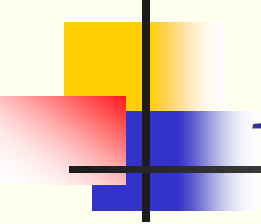


《软件工程》核心知识点 串讲 (SUMMARY)

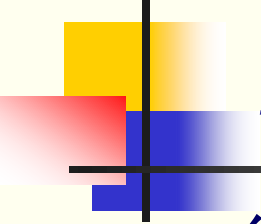
第一讲 概述





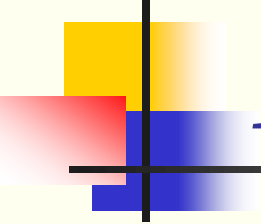
1.1 什么是软件?

- **软件=程序+数据+相关的文档**
- 专业化开发的软件包括：
 - 1) 能够提供客户所需功能与性能的**计算机程序**;
 - 2) 使程序能够适当的操作信息的**数据结构**
 - 3) 用于设置程序的**配置文件**、用于描述程序结构及开发过程的**系统文档**及解释如何使用系统的**用户文档**。



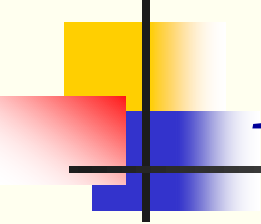
1.2 优良软件的属性

- 优良的软件应能交付相应的功能与性能，而且应具有良好的可维护性、可依赖性、有效性和可用性：
 - 可维护性 (Maintainability)，软件按需求变化不断进化的能力；
 - 可依赖性 (Dependability)，包括软件可靠性、保密性、安全性；
 - 有效性 (Efficiency)，软件对内存、CPU等系统资源的利用率；
 - 可用性 (Usability)，软件是否易用，易被客户接受。



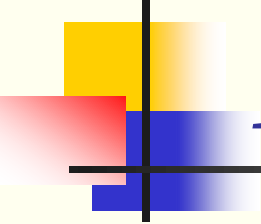
1.3 工程与工程活动

- 所谓**工程**，就是应用有关的**科学知识**与**技术手段**，通过有组织的**群体协作活动**，建造具有预期使用价值的**人造产品**的过程。如建筑工程、飞机制造等。
- 工程活动的共性特征：
 - 大规模设计与建造
 - 复杂问题与目标分解
 - 团队协作与过程控制



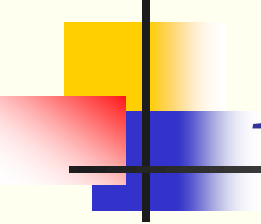
1.4 软件工程与软件危机

- **软件危机**是指落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列严重问题的现象，如开发效率低下、成本高、质量无法保障、可维护性差等。
- 软件工程被认为是能够解决软件开发严重问题的有效途径。



1.5 什么是软件工程？

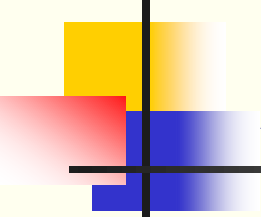
- 软件工程是涉及软件生产各个方面的一门工程学科
- 软件工程涉及软件生命周期的各个方面，从软件需求的确定到软件退役。
- 软件工程还涉及软件开发中的人为因素，如团队组织；经济因素，如成本估算；法律因素，如版权保护。



1.5 什么是软件工程？

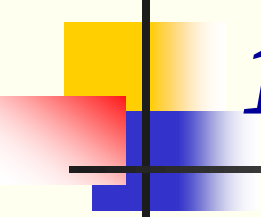
- 软件工程：（1）将系统化的、规范的、可度量的方法应用于软件的开发、运行和维护的过程，即将工程化应用于软件；（2）研究（1）中的方法。

——IEEE[IEE93]



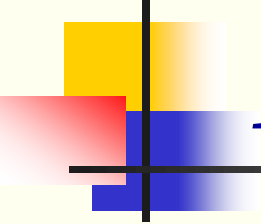
1.6 软件过程与软件生命周期

- **软件过程**是指开发或制作软件产品的一系列活动及其成果。
- 所有的软件过程中都包括四个基本活动：
 - **1. 描述**（ **Specification**） - 系统应该提供的功能及其开发约束；
 - **2. 开发**（ **Development**） - 软件产品的生产过程；
 - **3. 有效性验证**（ **Validation** ） - 检验软件产品是否满足了客户的需要；
 - **4. 进化**（ **Evolution** ） - 按照用户的变更要求不断的改进软件。
- **软件生命周期**是软件过程的另一种形象描述，通常包括需求定义、分析与描述、软件设计、实现、测试、维护与退役等活动。



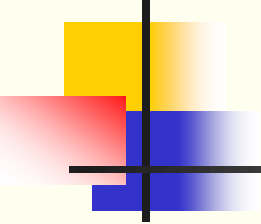
1.7 软件工程方法

- 软件工程方法是软件生命周期中使用的一整套技术方法的集合，包括系统模型描述、约束规则、设计建议和过程指南等组成元素。
- 目前，软件方法的发展经历了四个阶段：
 - **面向过程：**以算法函数为基本构造单元，强调自顶向下，逐层细化。
 - **面向对象：**以类（对象）为基本构造单元，对象间通过消息进行协作。
 - **面向构件：**以比类粒度更大、更易复用的构件为基本构造单元。
 - **面向服务：**在应用层次上把软件设计成可以互操作的服务。
- 四个阶段软件复用程度逐渐提升，软件开发效率越来越高



1.8 软件项目的成功要素

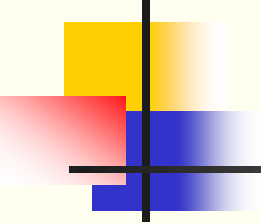
- 一个成功软件项目的三个要素包括：
 - 按时交付
 - 不超预算
 - 满足用户要求。



1.9 软件工程伦理问题与行业规范

- 软件工程伦理问题涉及社会、公众健康与安全、法律、环境、职业道德与规范等对于软件工程技术人员提出的要求与约束。
- 具有优秀的工程伦理素养、高尚的职业道德与责任，遵守行业规范要求是优秀软件工程师的基本素质。
- 软件开发者的职业道德行为不仅仅需要通过遵守法律来约束，更需要通过职业教育和团队培养促进软件工程师的良好职业美的形成。

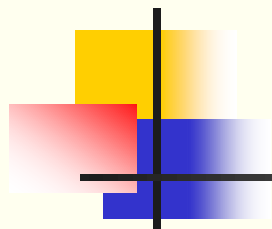




1.9 软件工程伦理问题与行业规范

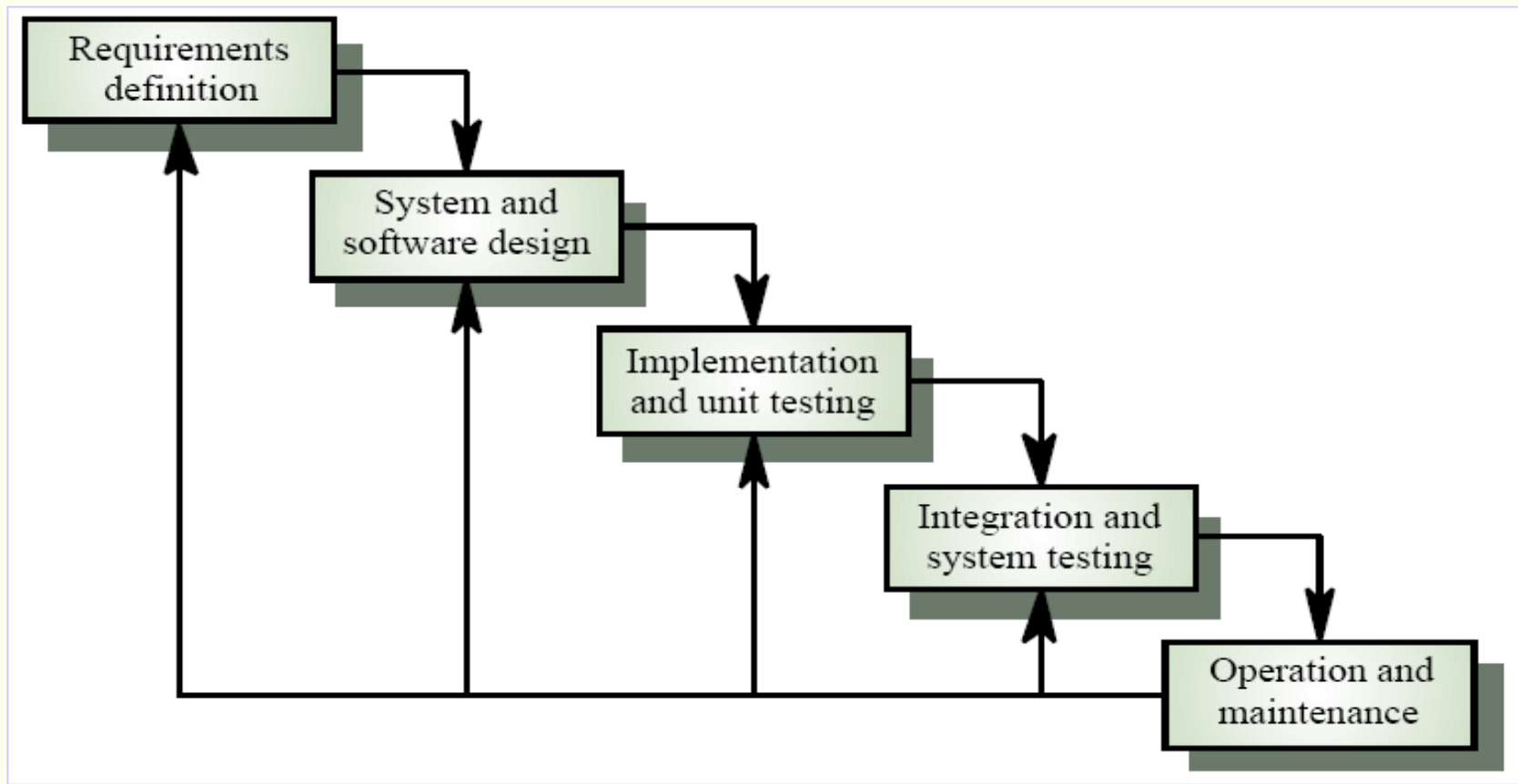
- 软件工程师应该遵循的行为准则：
 - **保密**，严守客户机密，不管是否签署了保密协议；
 - **工作能力**，如实表述自己的能力，不有意接收超出能力范围的工作；
 - **知识产权**，应知晓知识产权相关法规，确保客户相关权益受到保护；
 - **计算机滥用**，不能运用自己的技能滥用他人计算机，给他人带来损失。





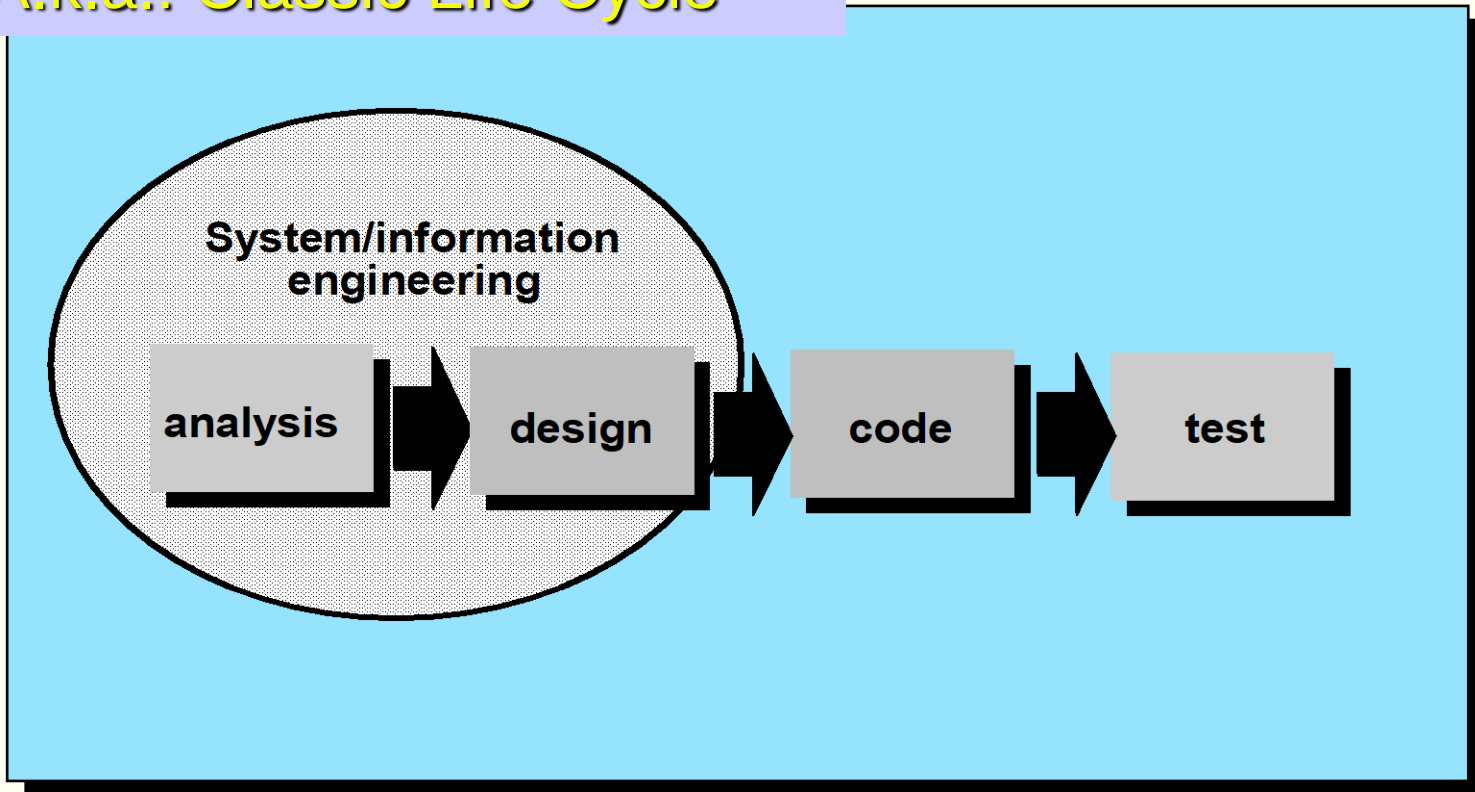
第二讲 软件过程

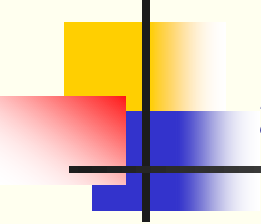
2.1 瀑布模型（顺序模型）



2.1 瀑布模型（顺序模型）

A.k.a.: Classic Life Cycle



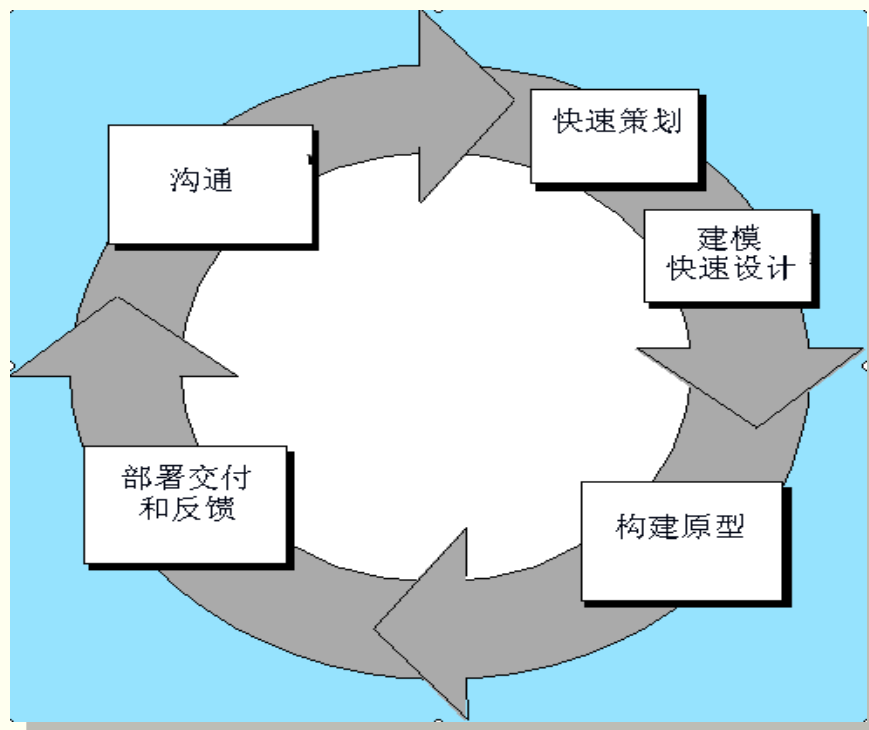


瀑布模型的特点和适用情况

- 瀑布模型起源于更一般的系统工程过程，反映了工程设计的基本思想。
- 这种模型生硬的把一个软件过程划分成几个界限清晰的阶段，而且这些阶段前后有严格的顺序，这导致它很难对用户的需求变更做出及时的调整；
- 因此，瀑布模型只适合需求非常清楚和需求变更被严格限制的情况下。

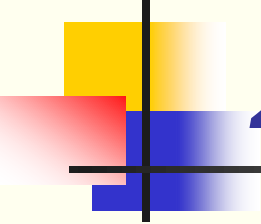
2.2 进化式开发模型

- **基本思想：**通过开发系统原型和用户反复交互，以明确需求，使系统在不断调整与修改中得以进化成熟。又叫做**原型式开发方法**。



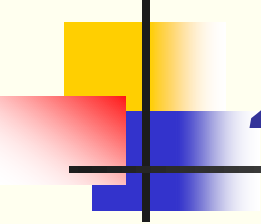
两种基本类型：

探索式开发；
抛弃式原型法。



2.2 进化式开发模型

- 较之瀑布模型有的三大优势：
 - 降低了适应需求变更的成本，变更分析与修改文档的工作量比瀑布模型少很多；
 - 用户的深度参与可以提供更及时有效的反馈，并能体现在不同版本的开发进度中；
 - 相比于瀑布模型，能够更早的交付和部署有用的软件给客户。



2.2 进化式开发模型

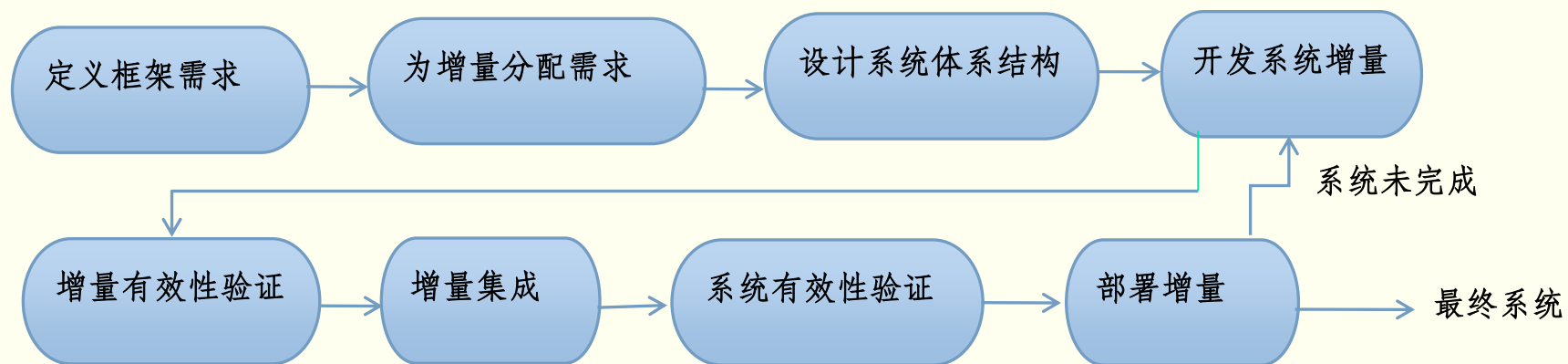
■ 问题

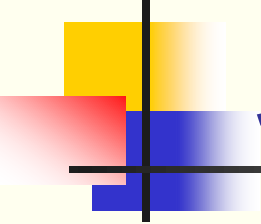
- 缺乏过程可见性;
- 系统结构通常会很差;
- 需要一些特别的技术（如原型快速开发技术），通常与主流技术不兼容.

■ 适用情况

- 适合中小规模的交互系统;
- 可用于大型系统的局部开发（如系统界面），可以和瀑布模型混合使用;
- 生命周期较短的系统。

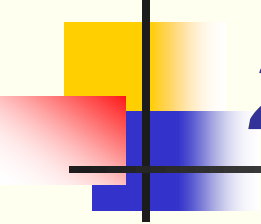
2.3 增量式开发





增量式开发的特点

- 在这种开发方式中，系统不是作为一个整体交付，而是被分解成若干个可以单独交付的模块--增量，每个增量交付系统的部分功能。
- 用户的需求按优先级排队，优先级最高的需求被放入最早交付的增量中。这样，优先级最高的系统功能就得到最多的测试，系统的可靠性较高。



2.4 基于构件的软件工程

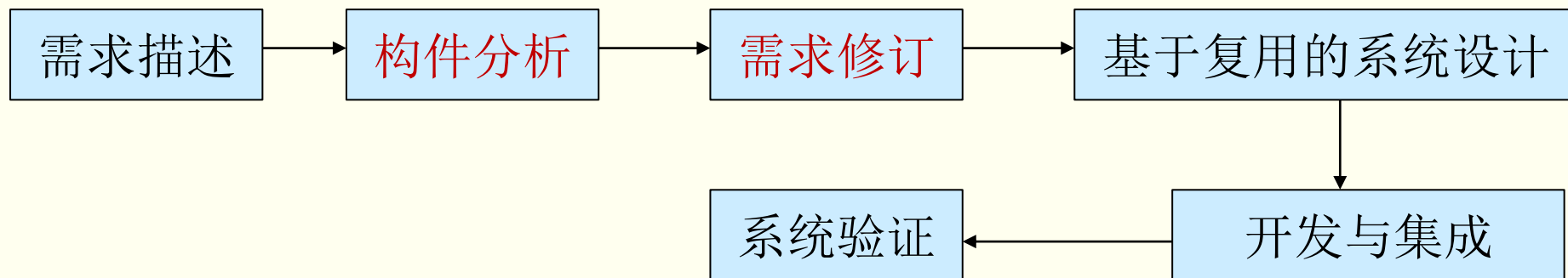
- 软件复用是指在两次或多次不同的软件开发过程中重复使用相同或相似软件元素（通常称为可复用构件、组件或软部件）的过程。
- 软构件是**标准的**、可以**互换的**、经过**装配**可随时使用的软件模块。

软件复用的意义

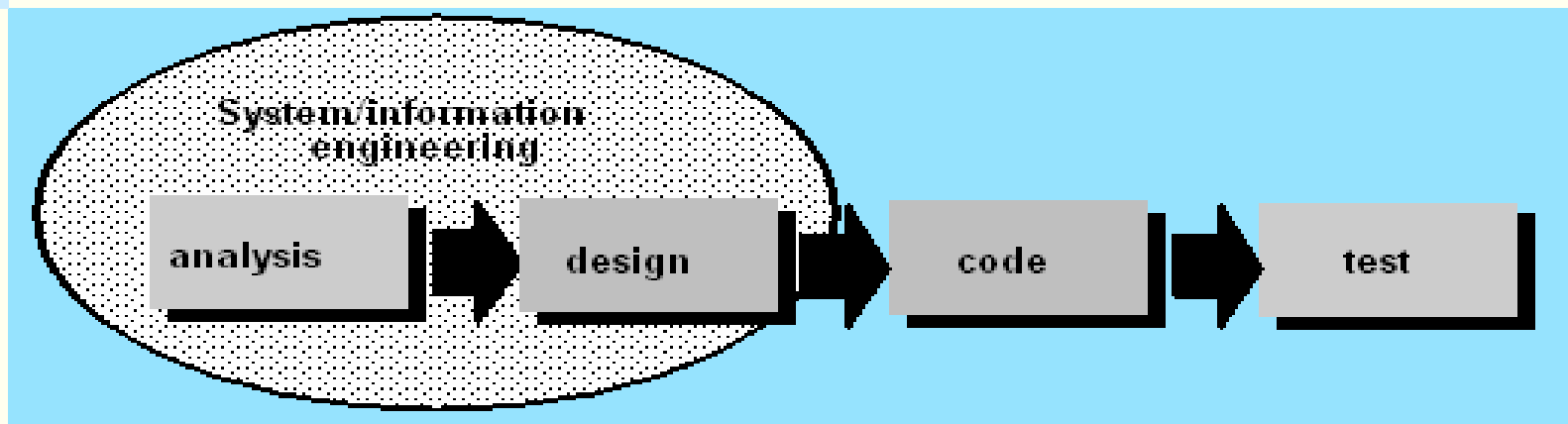
- 软件复用的出发点是使软件系统的开发不再“一切从零开始”，能够充分利用已有的知识和经验。
- 软件复用能够在软件开发中避免重复劳动，充分利用已有的开发成果，提高开发效率，降低开发成本。
- 软件复用还可以避免全新开发可能引入的错误，从而提高软件的开发质量。

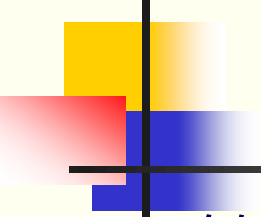


基于构件的软件工程



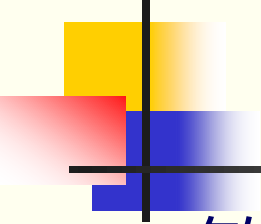
VS





2.5 敏捷软件开发

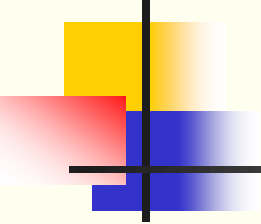
- 敏捷开发方法是20世纪90年代提出的一些支持快速交付的非重量级开发方法的统称，这类方法允许开发团队更关注软件本身而不是编制文档。
- 敏捷宣言：
 - 个体和交互胜过工具和过程；
 - 编写软件胜过书写详尽的文档；
 - 用户合作胜过合同谈判；
 - 响应变更胜过遵循计划。
- 极限编程和Scrum等敏捷开发方法都是建立在增量式开发和迭代开发的概念上，都基于敏捷宣言，基本原则相同。



2.5 敏捷软件开发

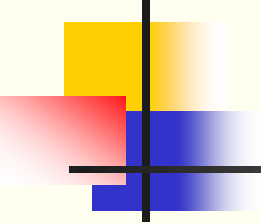
■ 敏捷开发方法的基本原则：

原则	描述
客户参与	客户应积极参与开发过程，负责提供新需求、需求排序和评估系统迭代。
增量式交付	软件以增量方式开发，由客户指定每个增量要包含的需求。
人非过程	开发团队的技术应得到承认与发扬，团队成员应该保持自己的工作风格。
接受变更	预料系统变更，并设计系统使之适应这些变更。
保持简单性	致力于所开发软件和开发过程的简单性，尽可能排除系统的复杂性。



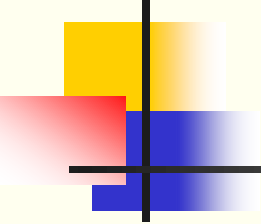
2.5 敏捷软件开发

- 敏捷开发方法的适用情况：
 - 中小规模的开发团队；
 - 每个团队成员都可以参与决策；
 - 需求变更频繁的项目；
 - 风险高的项目；



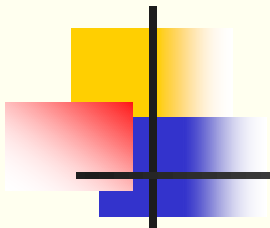
2.6 软件过程改进与CMM

- 软件过程改进是提升软件产品质量以及软件过程效率和效果的重要手段。
- 软件过程改进重点在于改善软件过程与项目管理以及将优良的软件工程实践引进到机构中，以达到提高软件质量、降低成本、减少开发时间的目的。
- 机构过程成熟度水平反映了机构在软件开发过程中采用优良的技术方法和管理实践的程度。
- 过程改进的主要目标是改善产品质量和过程可预测性。



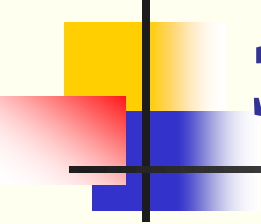
2.6 软件过程改进与CMM

- CMM (Capability Maturity Model for Software)，英文缩写名是SM-CMM，它指“软件能力成熟度模型”，CMM是美国卡内基—梅隆大学软件工程研究所（简称SEI）的研究成果；
- SEI给CMM下的定义是：对于软件组织在定义，实现，度量，控制和改善其软件过程的进程中各个发展阶段的描述。
- SW-CMM为软件企业的过程能力提供了台阶式结构，共分五级，分别是初始级、可重级、定义级、管理级和优化级。



第三讲 需求工程





3.1 需求分析基础

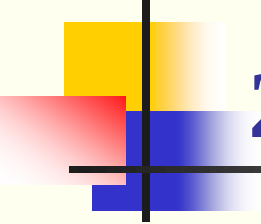
■ 1) 需求的两个不同层次^{层次}的描述

■ 用户需求

- 从客户的角度，采用^{自然语言}配合以图表对目标系统应提供的^{功能}以及系统操作要受到的^{约束}进行的声明。

■ 系统需求

- 系统需求是一种结构化文档，要运用一些^{专业的模型}详细的描述系统的^{功能}及其^{约束}。系统需求文档有时也称为功能描述，应该是精确的，它可以成为双方之间合同的重要内容，同时作为开发工作的依据。



2) 功能需求与非功能需求

■ 功能需求

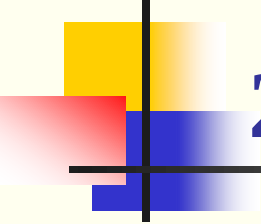
- 对系统应提供的功能，系统在特定的输入下做出的反应及特定条件下的行为的描述。某些情况下还要包括系统不应做什么。

■ 非功能需求

- 对系统提供服务或功能时受到的约束进行描述。如时间约束、开发过程约束和标准等。

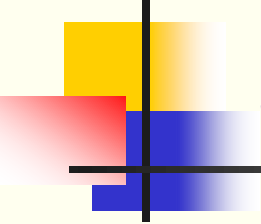
■ 领域需求

- 这种需求来自于系统的应用领域，反映领域特征。可能是功能需求也可能是非功能需求。



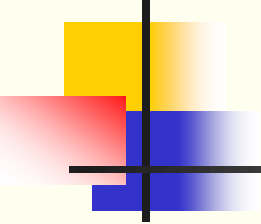
2) 功能需求与非功能需求

- 功能性需求与非功能性需求相比较，非功能需求往往更为关键，因为非功能需求表示的是系统的整体特征，而功能性需求描述的则是局部功能。
- 功能性需求描述应该具备全面性和一致性。
- 非功能需求的常见问题是检验困难，描述时应该设定明确目标并设定可测试的度量标准。



3) 使用自然语言描述需求的准则

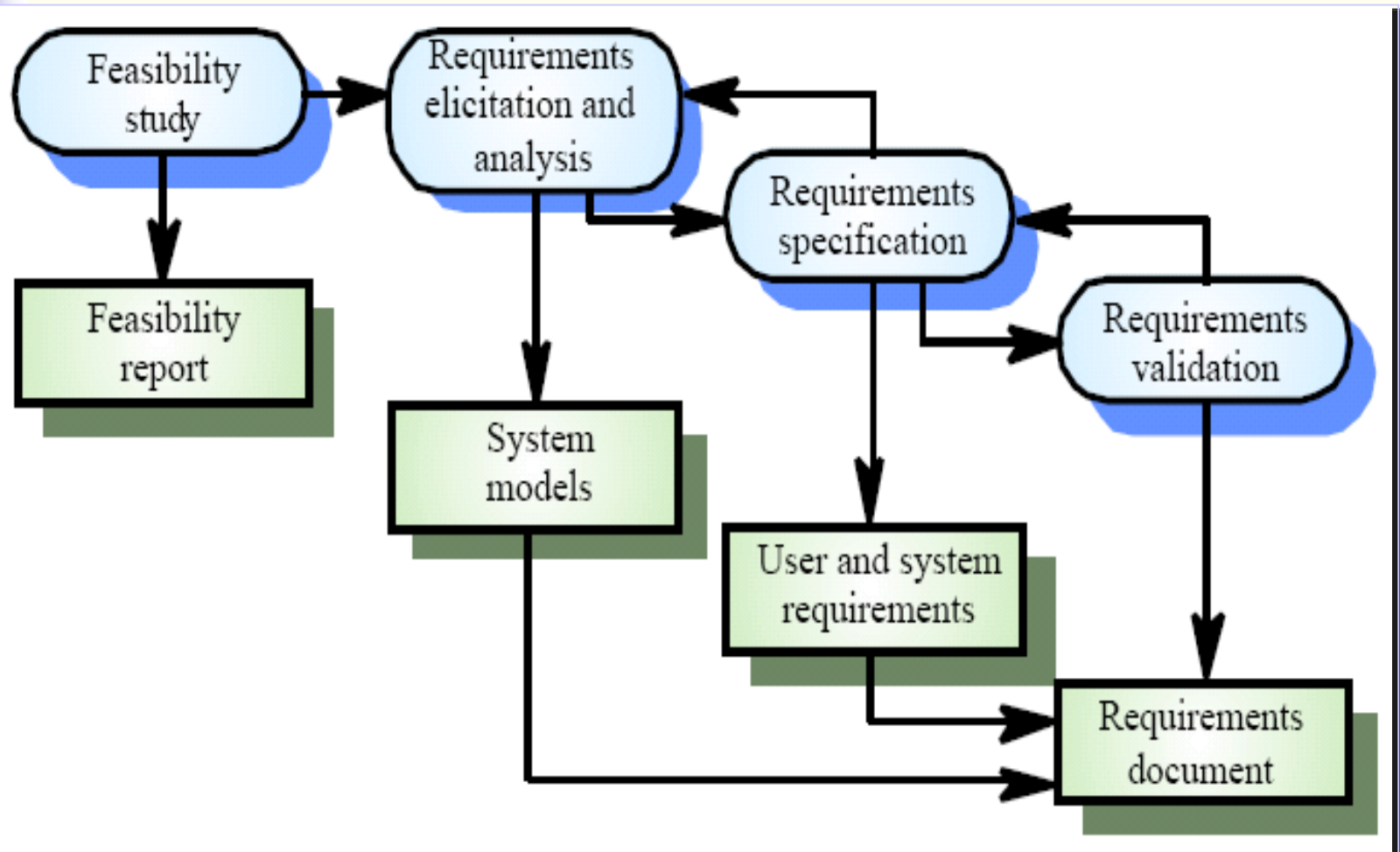
- 设计一个标准格式，以帮助减少遗漏，避免不必要的细节描述；
- 使用一致的语言，尤其强调区别强制性需求与希望性需求。如使用“必须”定义强制性需求，使用“应该”定义希望性需求；
- 使用文本加亮来突出关键性需求；
- 用户需求描述时尽量避免使用计算机专用术语；
- 尽可能把需求原理（需求产生的原因）和对应需求联系在一起。

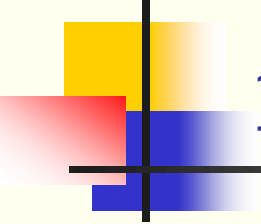


3.2 需求工程过程

- 需求工程过程并不具有唯一的模型，在所有的过程中都会涉及一些共同的活动，它们是：
 - 可行性研究；
 - 需求导出与分析；
 - 需求描述；
 - 需求有效性验证；
 - 需求管理。

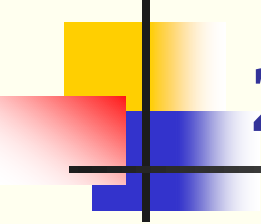
需求工程过程





1) 可行性研究

- 可行性研究要决定被提议的系统是否值得去做。
- 进行可行性研究包括 **信息评估**、**信息汇总**和 **书写报告**三部分工作。
- 信息的评估与汇总需要分析人员与**信息持有者**（Stakeholders）相沟通，通过提出问题汇总信息。



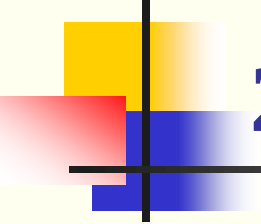
2) 需求导出与分析

- 这个阶段在可行性研究之后进行，*通常与需求描述交叉进行。*
- 需求导出的过程活动包括：*需求发现、需求的分类与组织、优先排序和冲突解决、需求文档化。*

2) 需求导出与分析

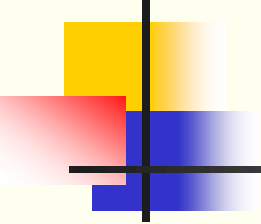
- **需求的发现与识别**是整个过程中**最为关键**的活动，负责收集目标系统级现存系统的相关信息并从这些信息中提炼出用户需求和系统需求。
- 信息的来源包括**已有的文件**，**系统的信息持有者**（**stakeholders**）以及**相近系统的规约描述**。





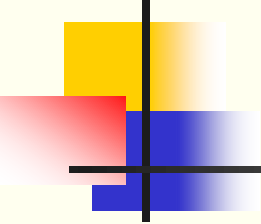
2) 需求导出与分析

- 需求要从多个视点进行分析
- **视点**用来表述不同角度的需求来源（信息持有者、其它相关系统及领域）。每一个视点代表系统需求的一个子集。
- 从多视点对系统进行分析是十分重要的，因为没有那一种单一的途径能够诠释整个系统需求。
- 视点一般可以分成**直接视点**、**间接视点**和**领域视点**。



3) 访谈

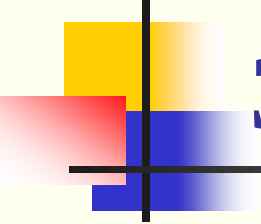
- 访谈是需求导出最广为使用的方法，对信息持有者（stakeholders）进行正式和非正式的访谈是需求工程过程最常见的工作。
- 有两种类型的访谈：
 - **封闭式访谈**，预先定义所要提问的问题，又叫做**限定式访谈**；
 - **开放式访谈**，不预先设定访谈内容，给定一个范围，与stakeholders自由交流以更好的把握他们的要求。
- 通常是采用两种方式相混合，以封闭式访谈开始，以开放式结束。



3) 访谈

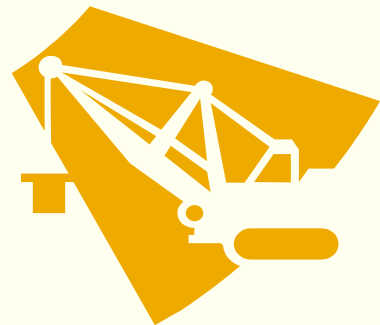
- 对于访谈者，要求应该是思维开阔的，愿意聆听他人的想法，并且不会对问题有先入为主的想法。
- 他们善于用问题和建议去启发访问者的思维，或使用原型与被访问者讨论，而不是仅用 ‘what do you want’ 进行简单的提问。





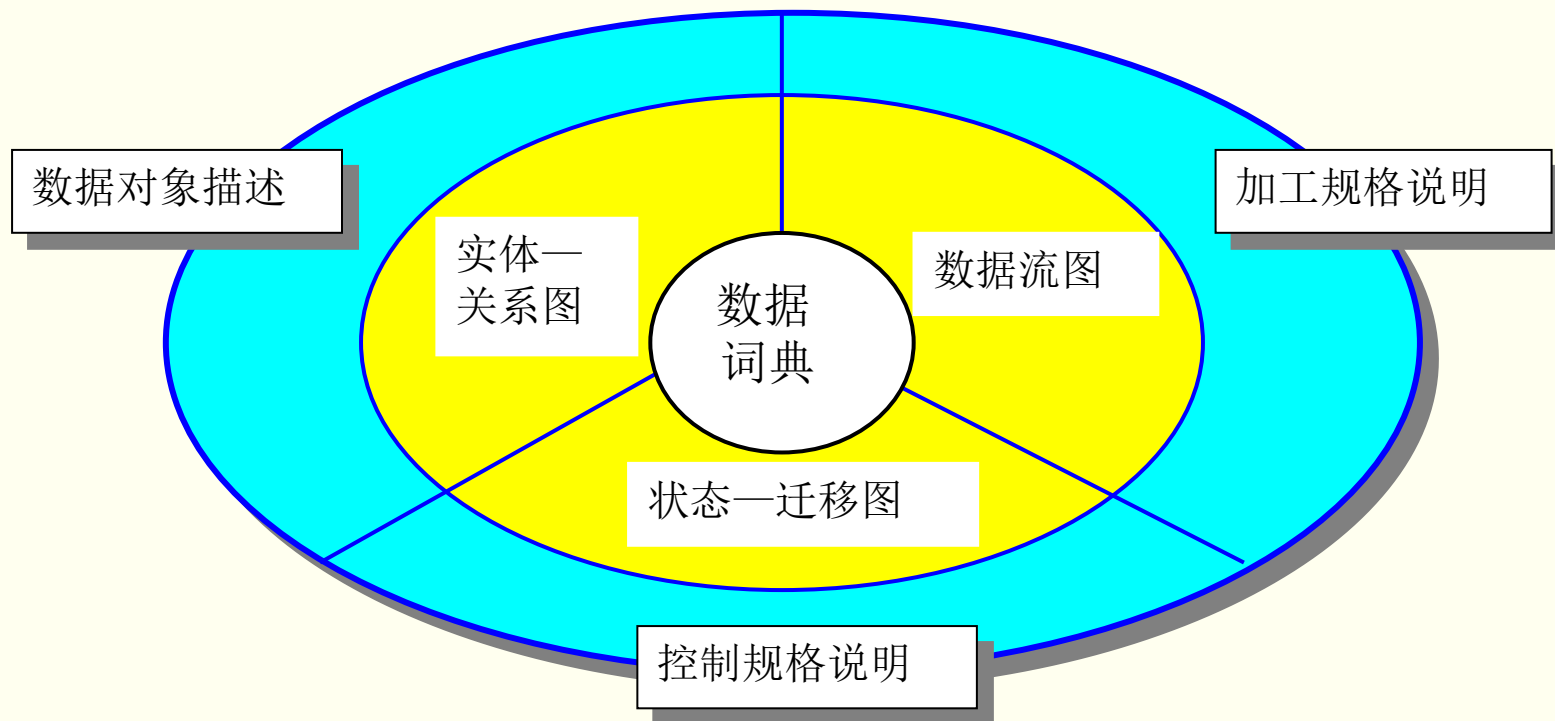
3.3 结构化分析（SA）建模

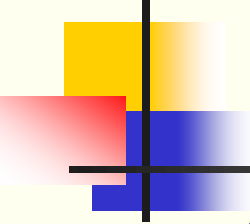
- 结构化分析方法是一种**面向数据流**的系统建模技术，它从数据加工的角度对系统进行规格描述；
- SA帮助分析者理解系统的功能，并采用模型与用户进行交流；
- 不同的模型从不同的角度对系统进行描述。



结构化分析建模

- 结构化分析方法建立的分析模型结构如下图：

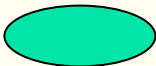
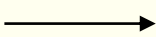


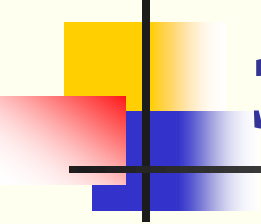


结构化分析建模

- 结构化分析模型的核心是**数据词典**，它描述了所有的在目标系统中使用的和生成的数据对象。
- 围绕着这个核心的有三种图：**实体—关系图**(ERD)描述数据对象及数据对象之间的关系；**数据流图**(DFD)描述数据在系统中如何被传送或变换，以及描述如何对数据流进行变换的功能（子功能）；**状态—迁移图**(STD)描述系统对外部事件如何响应，如何动作。
- 因此，ERD用于数据建模，DFD用于功能建模，STD用于行为建模。

数据流图的画法

- 数据流图中的常用图元有以下四种：
-  表示外部实体，代表数据源和数据池（终点）。
-  表示加工，代表接收输入，经过变换，继而产生输出的处理过程。
-  表示数据流，代表数据的流向和路径。
-  表示数据存储，代表系统加工的数据所存储的地方。

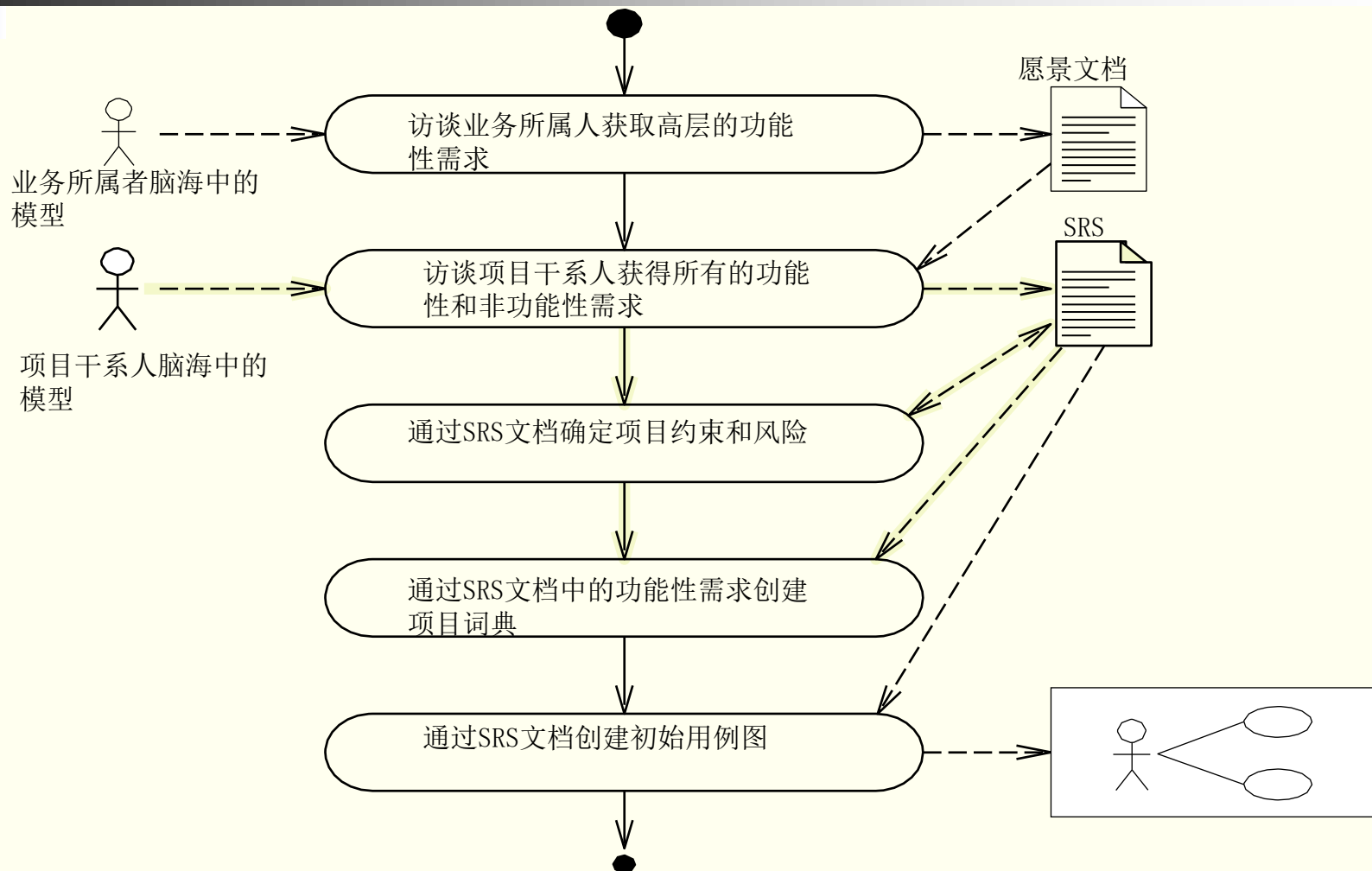


3.4 UML与面向对象分析方法

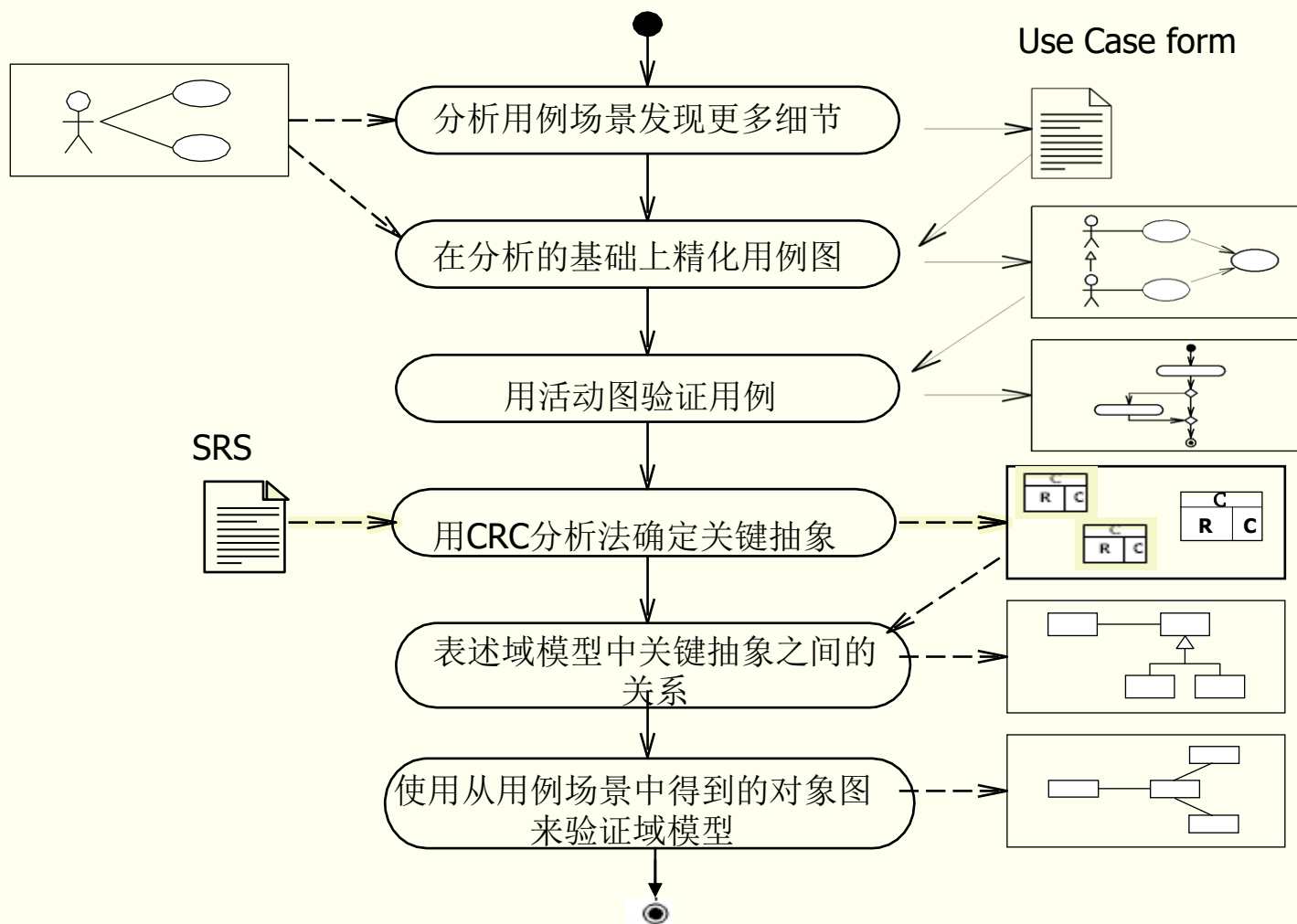
3.4.1 理解UML

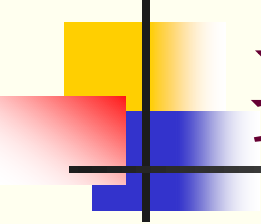
- UML是一种**标准的图形化建模语言**，它为不同领域的人们**提供一种统一的交流标准**，这种标准使得系统构造者能够用标准的、易于理解的方式建立能表达出他们想象力的系统蓝图，并使客户、分析员、设计人员、程序员和系统其它涉及者能够相互理解和达成一致，从而能够有效地共享和交流设计结果。

3.4.2 需求导出工作流程



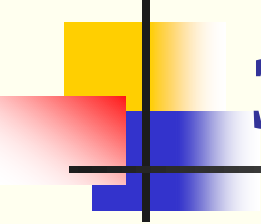
3.4.3 需求精化工作流程





要求掌握：（结合课件与相关练习复习）

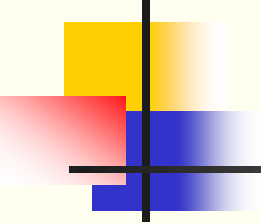
- 1) 用例图的画法；
- 2) 用例表（用例规约描述）的基本结构及描述方法；
- 3) 活动图的画法
- 4) 用CRC确定关键抽象的过程；
- 5) 用分析类模型表示关键抽象。



3.5 需求有效性验证

- 需求有效性验证的目的是检验需求描述是否正确地反映了客户的意愿。
- 好的需求对软件系统的开发效率及软件质量起着至关重要的作用。一个错误发现的越晚，修改它所付出的代价就越大。
 - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

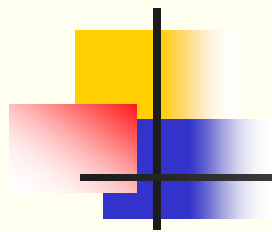
阶段	需求分析	软件设计	程序编码	单元测试	验收测试	维 护
相对修复代价	0.1~0.2	0.5	1	2	5	20



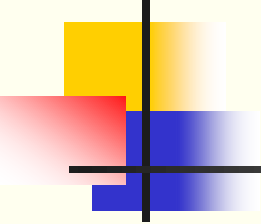
需求检查

对需求文档中定义的需求要进行多种检查，
这些检查包括：

- **有效性**， Does the system provide the functions which best support the customer's needs?
- **一致性**， Are there any requirements conflicts?
- **完备性**， Are all functions required by the customer included?
- **现实性**， Can the requirements be implemented given available budget and technology?
- **可检查性**， Can the requirements be checked?



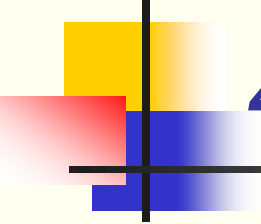
第四讲 设计工程



4.1 软件工程中的设计

- 设计是一个把问题转换成解决方案的创造性过程；
- 设计解决的是“**如何实现系统**”的问题；
- 从工程管理的角度，软件设计可以分成**概要设计**
（总体设计、系统设计）与**细节设计**（详细设计）

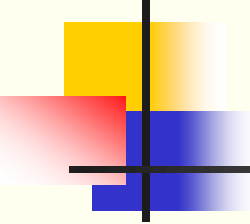




4.2 设计概念

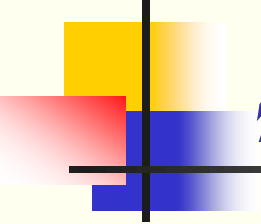
4.2.1 模块化

- 模块化的思想，即把软件划分为可独立命名和编址的构件，每个构件称为一个**模块**，每个模块完成一个子功能，当把所有模块组装到一起成为一个整体时，便可以完成指定的功能。
- 模块组成系统或子系统。
- “一个复杂问题分割成若干个容易解决、容易管理的小问题后更易于求解”，模块化正是以此为依据把系统划分成若干个模块，各个击破。



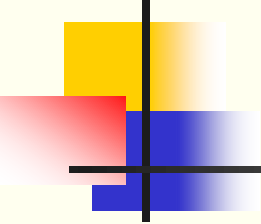
4.2.2 信息隐藏与独立性

- **信息隐藏原理**告诉我们，模块应该设计得使其所含信息（过程和数据）对于那些不需要这些信息的模块来说不可访问；每个模块只完成一个相对独立的特定功能；模块之间仅交换那些为完成系统功能必须交换的信息，即模块应该功能**独立**的。
- **采用信息隐藏原理指导模块设计有很多好处：**
 - 1) 它支持模块的并行开发；
 - 2) **减少测试和后期维护的工作量。**因为测试和维护阶段不可避免地要修改设计和代码，模块对大多数数据和过程处理细节的隐藏可以减少错误向外传播。
 - 3) **整个系统扩充功能只需“插入”新模块，**原有的多数模块无须改动。



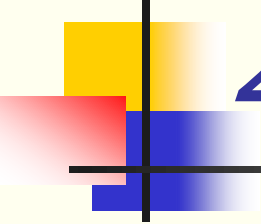
模块独立性 (*Independency*)

- 模块独立性的概念是模块化、抽象和信息隐藏概念的直接产物，模块独立性是通过开发具有单一功能和与其他模块没有过多交互作用的模块来达到的。
- 独立性好的模块对其它的模块依赖性小，修改时对其它模块的影响小，易于修改和扩充，因此有良好的可维护性。
- 模块独立性可用两个定性准则来度量：**耦合性** (coupling) 和**内聚性** (cohesion) 。
- **耦合**是模块之间相对独立性的量度，而**内聚**则是模块功能相对强度的量度。
- **模块的内聚性越强，耦合性越弱，独立性越强。**



4.2.3 抽象

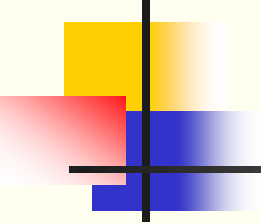
- 人类在认识复杂现象的过程中使用的最强有力的思维工具就是**抽象**。
- **抽象就是抽出事物的本质特性而暂时不考虑它们的细节。**
- 一个复杂的动态系统首先可以用一些高级的抽象概念构造和理解，这些高级概念又可以用一些较低级的概念构造和理解，如此进行下去，直至最低层次的具体元素。



4.2.4 逐步求精 (细化)

“逐步求精”是与“抽象”密切相关的一个概念，它由N.Wirth提出，可视为一种早期的**自顶向下设计策略**，其**主要思想**是：针对某个功能的宏观描述，用逐步求精的方法不断地分解，逐步确立过程细节，直至该功能用程序语言描述的算法实现为止。

因为求精的每一步都是用更为详细的描述替代上一层次的抽象描述，所以在整个设计过程中产生的、具有不同详细程度的各种描述，组成了系统的层次结构。层次结构的上一层是下一层的**抽象**，下一层是上一层的**求精**。

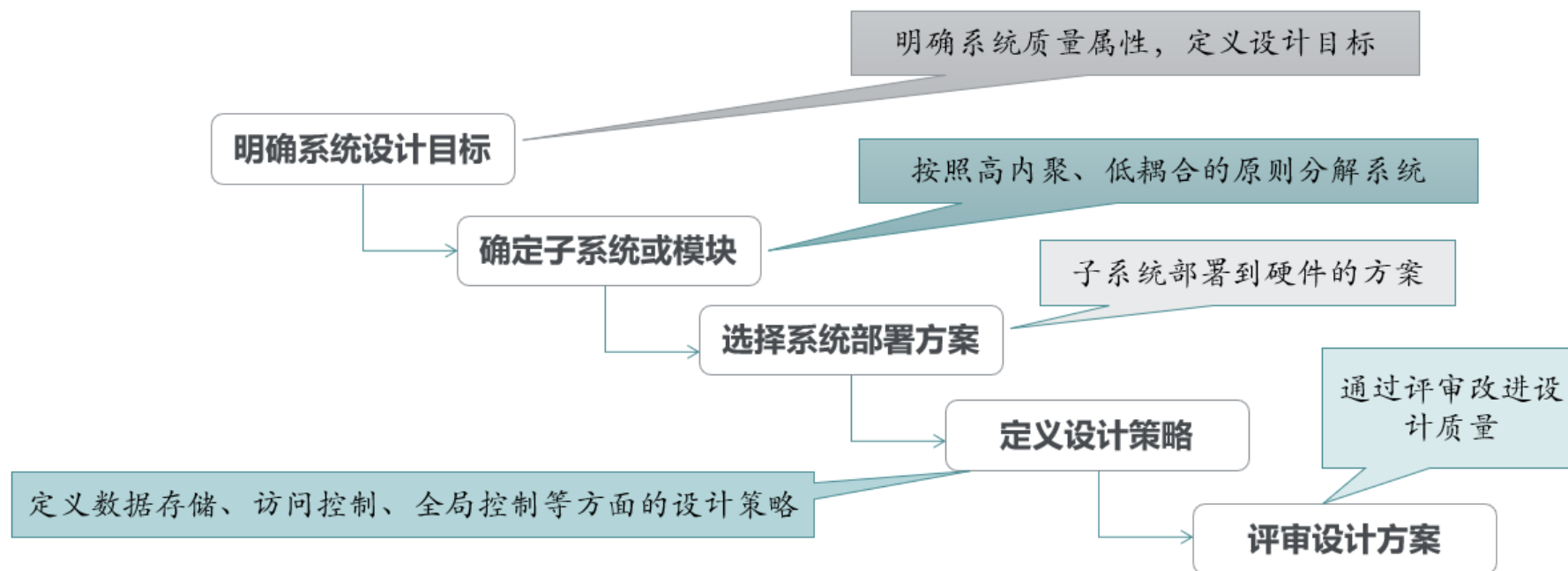


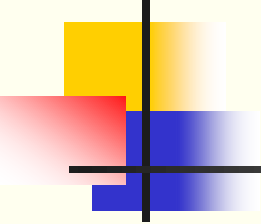
4.3 体系结构设计

- 体系结构（architecture, 又称架构）设计的任务是要识别出组成系统的子系统并建立子系统的控制和通信框架。
- 体系结构设计是联系需求描述与其他设计活动的桥梁。



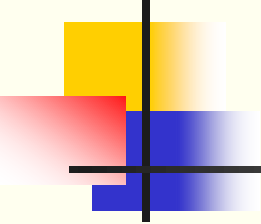
4.3.1 体系结构设计过程活动





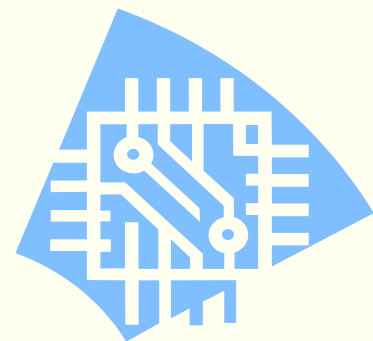
4.3.2 体系结构有关的几个概念

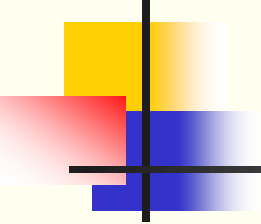
- **体系结构风格**：用于描述某一特定应用领域中系统组织的惯用模式，反映了领域中众多系统所共有的结构和语义特性。
- **设计模式**：描述了软件系统设计过程中常见问题的一些解决方案，通常是从大量的成功实践中总结出来的且被广泛公认的实践和知识。
- **软件框架**：软件框架是由开发人员定制的应用系统的骨架，是整个或部分系统的可重用设计，由一组抽象构件和构件实例间的交互方式组成。



4.3.3 几种常见的体系结构风格

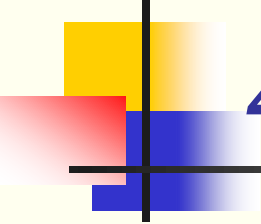
- 系统的体系结构风格反映的是系统组织所采用的基本策略。
- 三种应用广泛的组成模型：
 - 数据中心体系结构（容器模型）；
 - 客户/服务器体系结构；
 - 抽象机或分层体系结构。





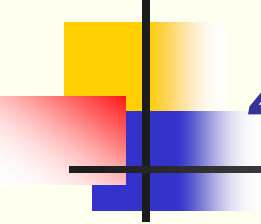
4.3.2.1 数据中心体系结构（容器）

- 数据中心体系结构（容器模型）的基本特点：
 - 所有共享数据放到一个中心数据库（容器）中，所有子系统都能从中存取数据；
- 当系统中存在大量的数据共享时，数据中心（容器）模型是最为常用的体系结构风格。



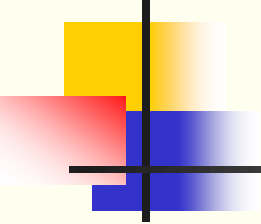
4.3.2.2 客户/服务器体系结构

- 客户/服务器模型是一个分布式系统模型，数据和加工过程在多个处理器之间分配；
- 这种模型的主要组成：
 - 一组为其它子系统提供服务的单机服务器；
 - 一组向服务器请求服务的客户机；
 - 连接客户机与服务器的网络。



4.3.2.3 分层（抽象机）体系结构

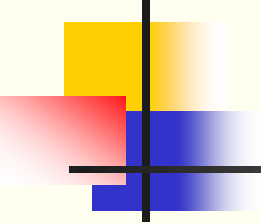
- 这种模型把系统组织成一系列的层次（抽象机），每一层提供一组服务；
- 这种模型支持增量式的开发，不同层次的服务可以单独交付；
- 层与层之间以接口相联系，一个接口发生改变，只有毗邻的层会受到影响；



4.4 体系结构视图

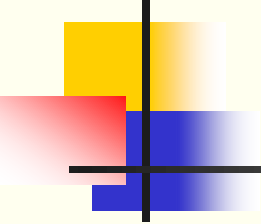
- 软件体系结构是对系统构成与部分间联系的多角度描述，每一种模型只能描述系统的一个侧面（视角）；
- 一个**体系结构视图**是对于从某一视角或某一点上看到的系统所做的简化描述，描述中涵盖了系统的某一特定方面，而省略了与此方面无关的实体。





4.5 控制模型

- 控制模型考虑子系统之间的控制流，这是分解模型不考虑的问题。
- 对控制流建模有两种一般性的方法：
 - **集中式控制**
 - 一个子系统专门负责控制，控制其他子系统的启动与停止。
 - **基于事件的控制**
 - 不将控制信息集中在一个子系统内，每个子系统都能够接受来自系统外部的事件并作出响应。

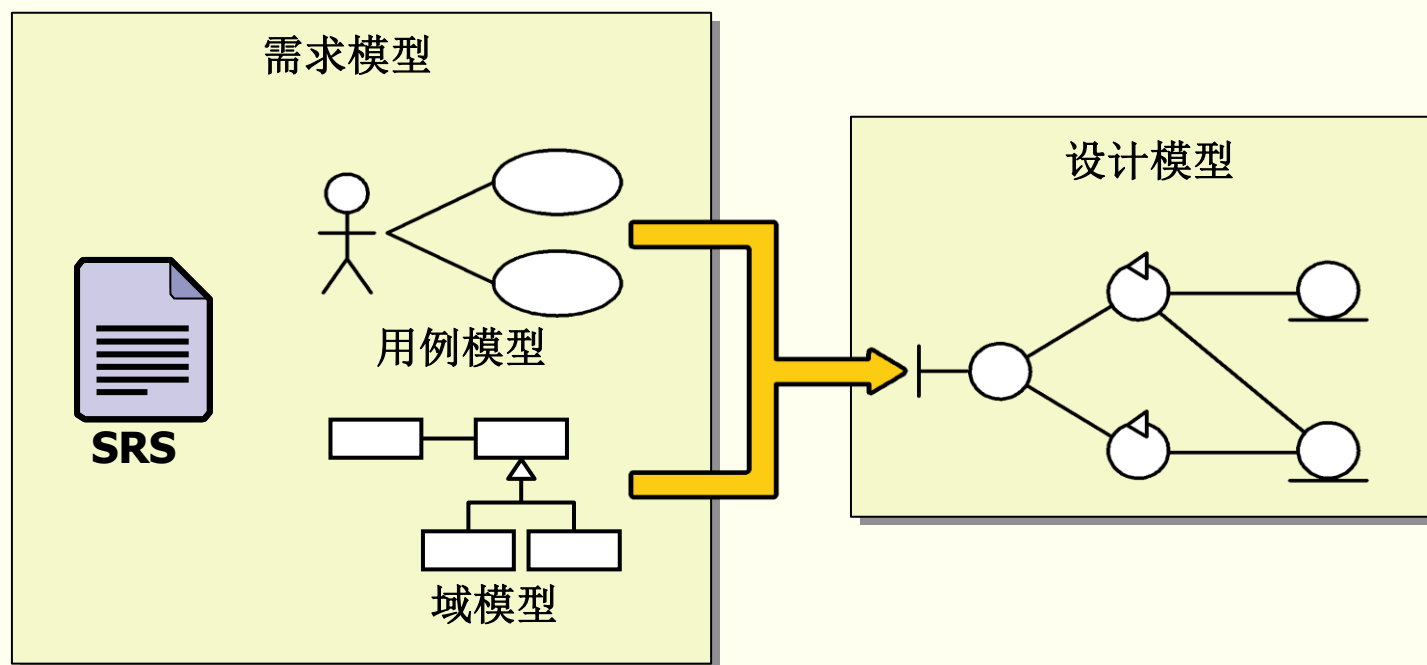


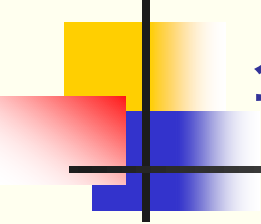
4.5 控制模型

- **集中式控制**按子系统顺序执行还是并行执行分为两类：
 - 调用-返回模型
 - 管理者模型
- **基于事件的控制**主要有两种：
 - 广播模型
 - 中断驱动模型

4.6 从分析到设计的转换——鲁棒性分析

鲁棒性分析是这样一个过程，它采用鲁棒图引导我们从用例转换为支持用例实现的职责模型：





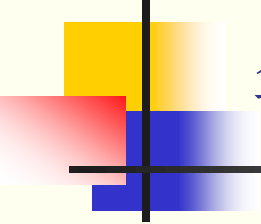
鲁棒性分析

鲁棒性分析的输入：

- 一个用例
- 这个用例的用例场景
- 这个用例的活动图（如果可以用到）
- 域模型（domain model）

鲁棒性分析的输出：

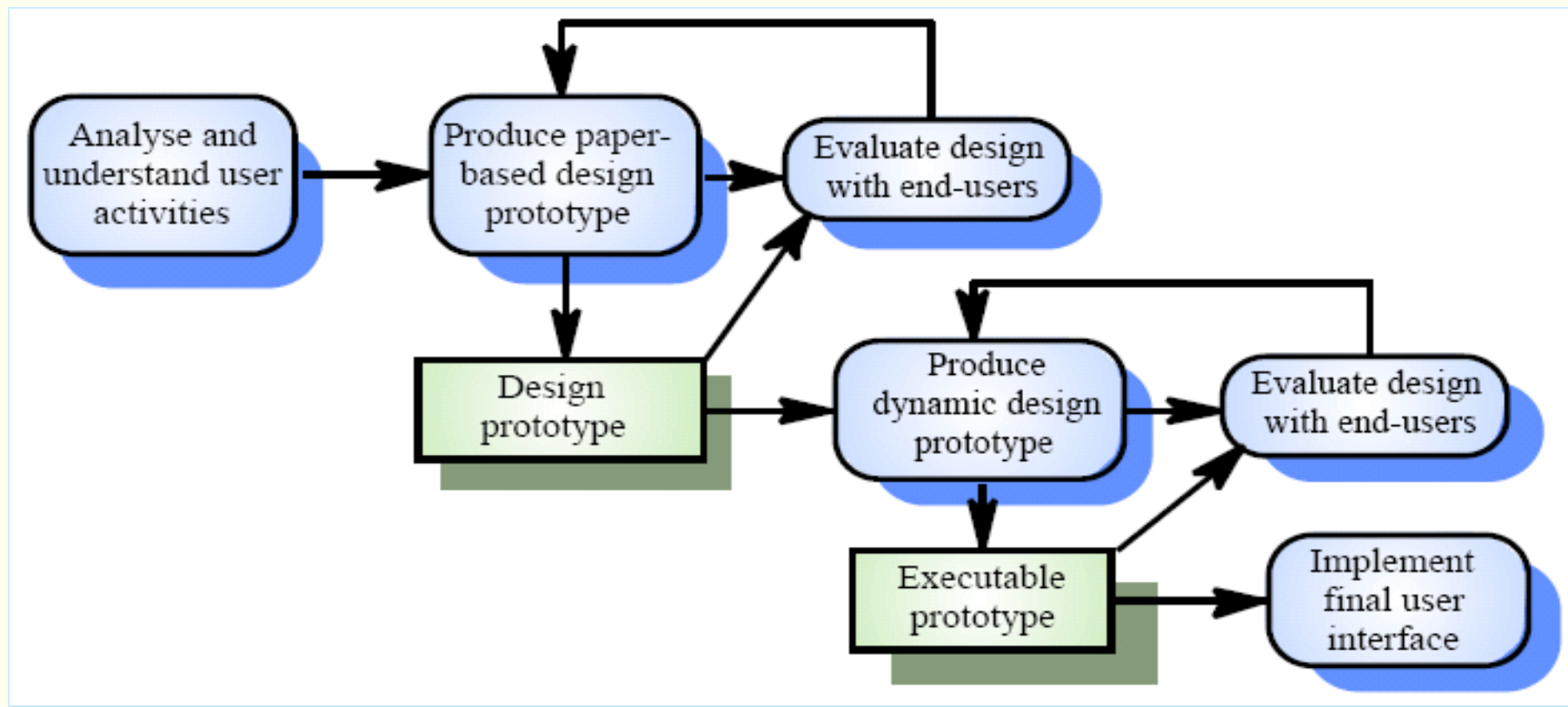
通过一个UML序列图和一些设计组件：边界、服务、实体组件，得出用鲁棒图表示的设计模型。

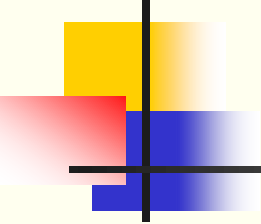


鲁棒性分析建立设计模型的过程

1. 选择一个适当的用例。
2. 把一个参与者放到**协作图**里面。
3. 分析这个用例（活动图）。
 - 对于用例的每一个动作：
 - a. 确定并增加边界组件
 - b. 确定并增加服务组件
 - c. 确定并增加实体组件
 - d. 画出这些组件间的关联
 - e. 把每个组件都贴上用来满足用例交互的动作标签
4. 把协作图转换成**序列图**（**结合课上练习与作业复习**）

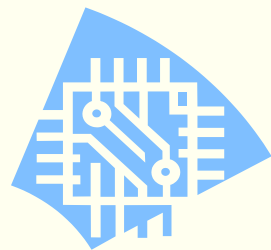
4.6 用户界面设计过程

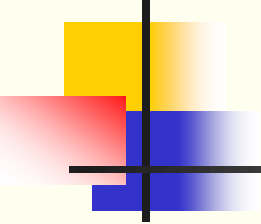




4.7 界面设计的一般原则

- 用户熟悉;
- 一致性;
- 意外最小化;
- 可恢复性;
- 用户引导;
- 用户差异性

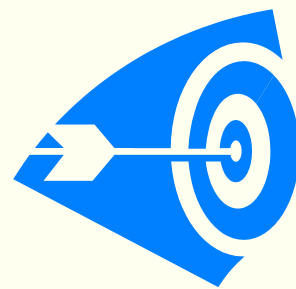


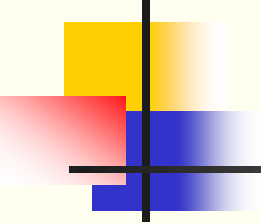


4.8 错误消息

- 错误消息的设计对界面设计的成败是非常关键的。设计很差的错误消息会导致用户对整个系统反感。
- 错误消息应该是礼貌的、简明的、一致的和建设性的。

(结合本章课件例题，要求能够分析不同错误消息提示的优劣，并能够设计符合上述原则的消息提示)

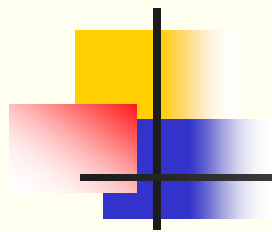




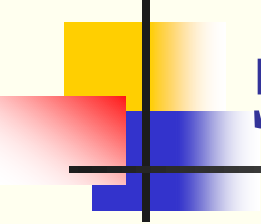
4.9 帮助系统设计

- 软件帮助系统**不能**是用户手册的简单复制，应该有一个合理的组织与结构，应该为用户提供不同的入口。



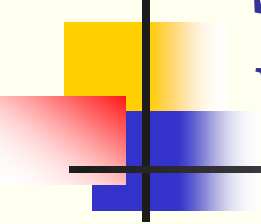


第五讲 软件实现与验证



5.1 程序设计与调试

- 程序设计的任务是把设计转换成程序以及在程序中去掉错误，包括编程与调试两个过程。
- 通常，程序员要对自己开发的程序进行测试，这时程序中的一些明显的错误会暴露出来并被根除，这个过程叫调试。



5.2 验证和有效性确认 (*Verification & Validation*)

- 验证:

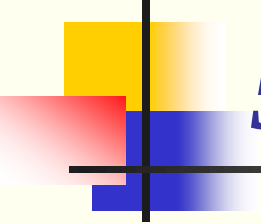
“Are we building the product right? ” .

- 检查软件是否符合它的规格描述。 (不符合描述的即为“软件缺陷”)

- 有效性确认:

“Are we building the right product? ” .

- 检查软件是否满足客户的期待。 (要检查系统是否符合客户的真实愿望)



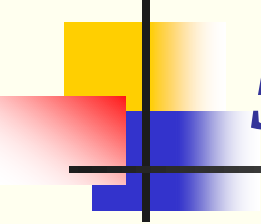
5.3 用来实现V&V 目标的测试类型

■ 缺陷测试

- 目标是找出软件中的缺陷和不足；（发现程序与规格描述之间的一致性）
- 一个成功的测试就是找出了系统存在某个目前未知的缺陷。

■ 有效性测试

- 目标是向开发者和用户证实系统满足了用户需求；
- 每个需求至少要有有一个测试，以证明系统是按预期运行的。（这种测试要反映真实的用户输入及其频率）



5.4 V&V 过程的两种基本方法:

- **软件审查**

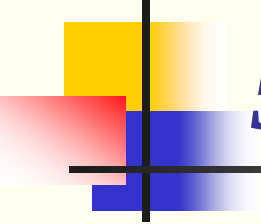
通过对系统的各种静态成果，如需求文档、设计文档、源代码，进行检查和分析发现问题。

- May be supplement by tool-based document and code analysis

- **软件测试**

通过使用测试数据执行系统，检查运行结果来发现问题。

- The system is executed with test data and its operational behaviour is observed

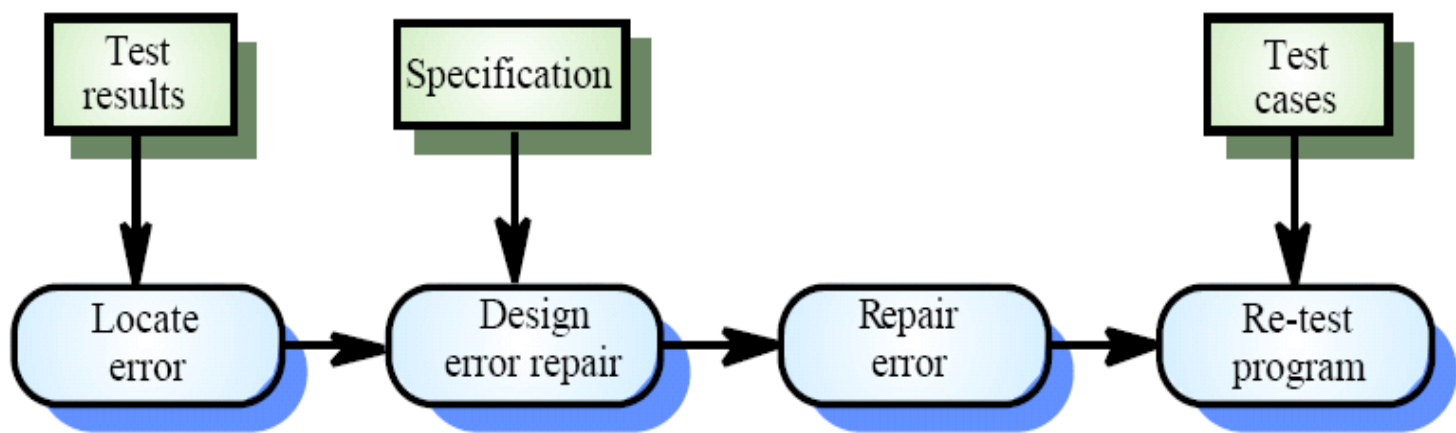


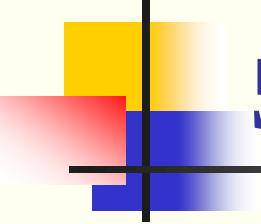
5.4 V & V 过程的两种基本方法:

- 测试的**目的**是为了**揭示程序中存在错误**，而不是没有错误。
- 按照测试的不同目标可以把测试分成**有效性测试**与**缺陷测试**。
- 静态审查无法检验软件是否可用，也不能检验非功能需求，因此**程序测试**是必不可少的，是起决定性作用的**V & V**技术。
- 在**V & V**过程中，**程序测试**和静态审查通常是结合在一起使用的。

测试和调试

- 测试和调试是不同的过程，通常交叉进行。
 - 缺陷测试的目的是确定系统中存在缺陷；
 - 调试考虑的是定位和修改缺陷。

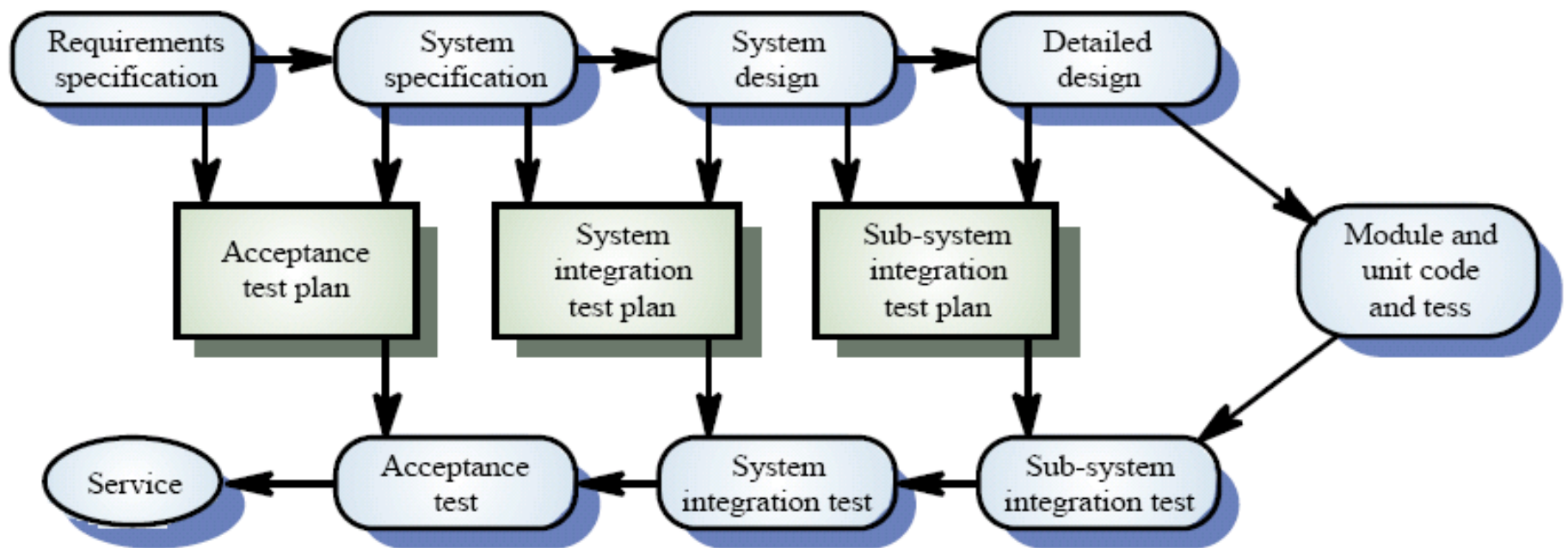




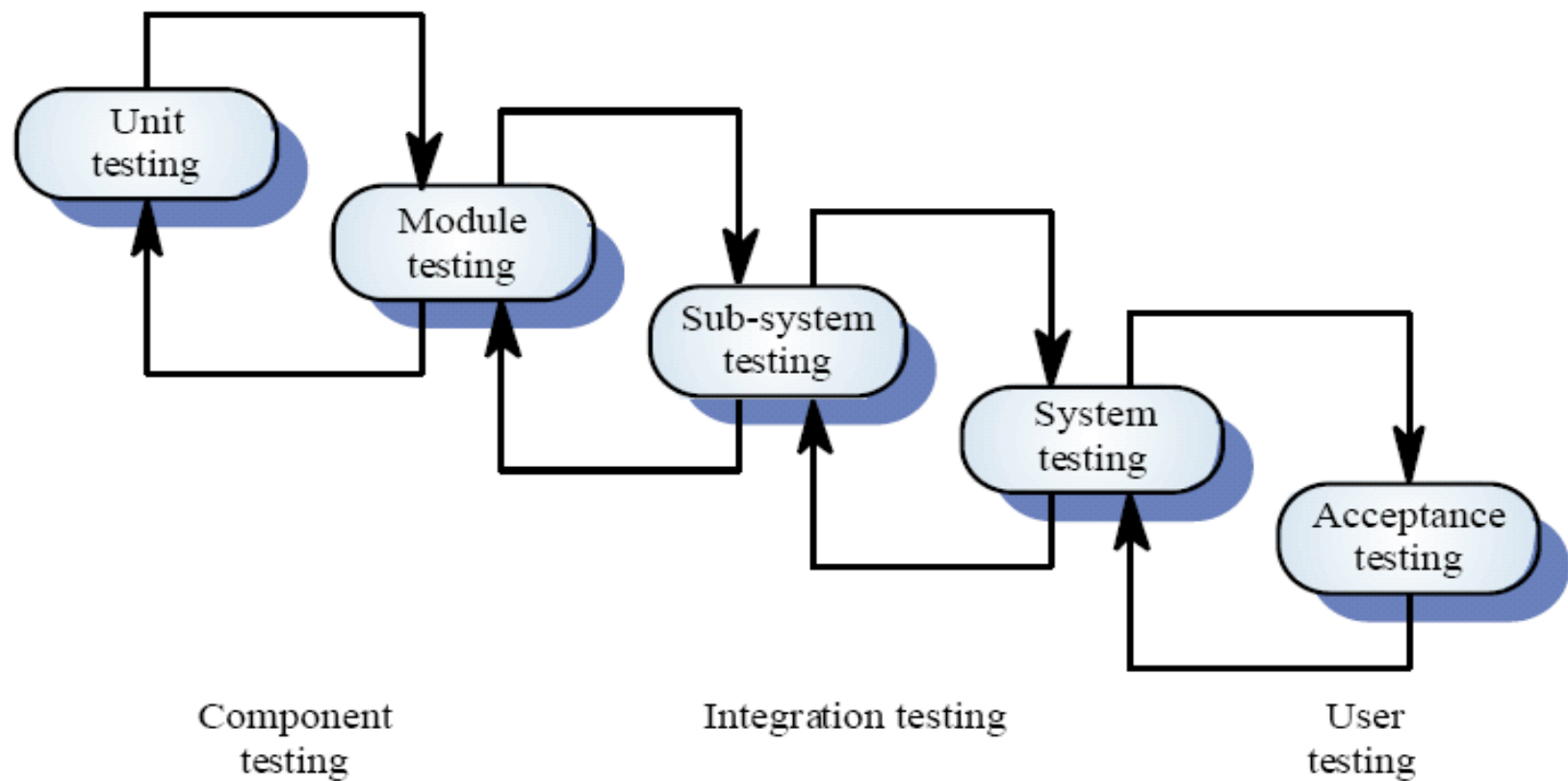
5.5 V & V 规划

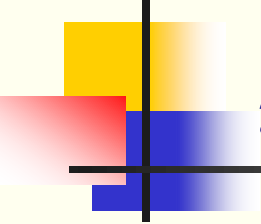
- 仔细的规划能够使程序检查和测试的工作得到更多的回报。
- V & V过程的规划应该从开发过程的早期就开始。
- V & V规划应该明确的说明静态检查与测试任务与分工。
- 测试规划主要是制定测试过程标准，而不是描述测试本身。

系统开发的V模型



5.7 软件测试阶段活动





测试阶段

■ 组件测试

- 测试单个的程序组件；
- 通常由程序开发者完成（除了要求特别高的系统）；
- 这个阶段的测试大多依靠测试者的经验。

■ 系统测试

- 测试由组件整合成的子系统和系统；
- 有专门的测试团队进行测试；
- 测试要依据需求规格说明进行。

5.8 集成测试

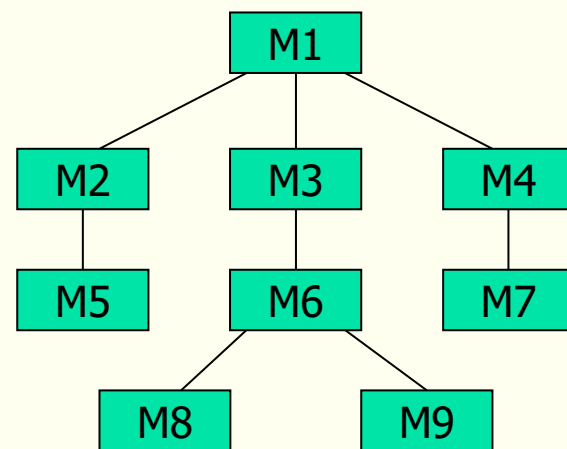
集成测试包括把组件集成为系统和对合成的系统进行测试，以发现组件集成过程带来的问题，集成方式可以分为：

■ 自顶向下集成

- 从主控模块开始，沿着控制层次结构逐步向下，利用深度优先或广度优先的方式将从属于主控模块的其他模块集成到系统结构中。

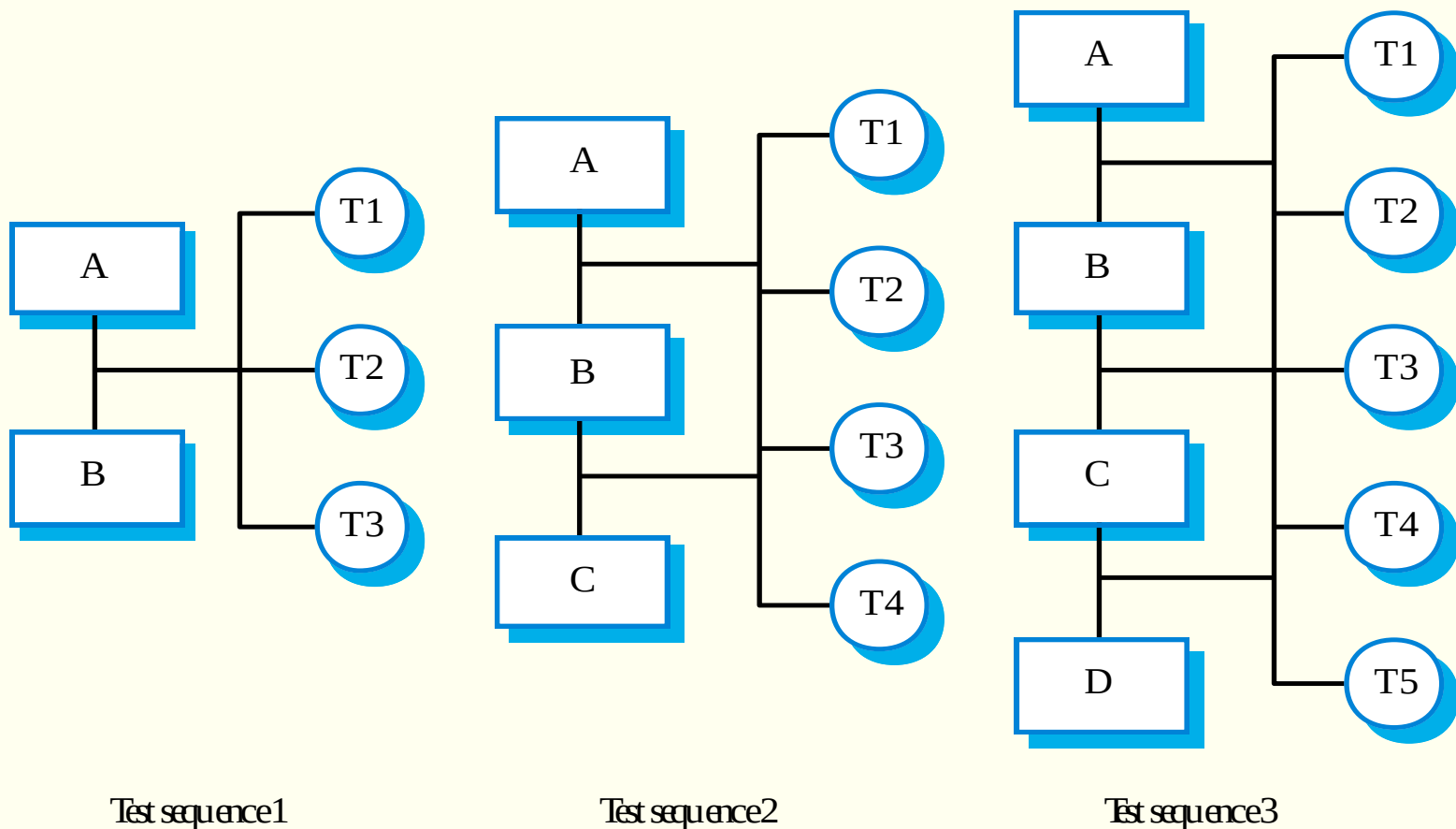
■ 自底向上集成

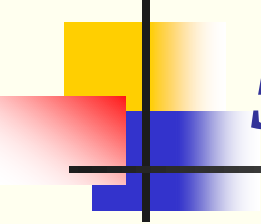
- 从原子模块开始，从底层把模块逐步向上集成为更大规模的子系统和系统。



增量集成测试

为了简化测试中错误定位的问题，可以采用增量集成的方法。

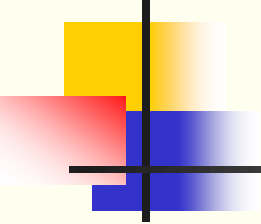




5.9 测试用例设计

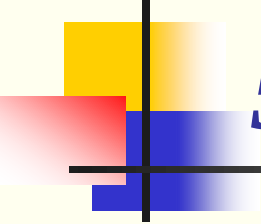
- 测试用例的基本构成可以包括：设计的输入、期望的输出、测试环境和测试对象的描述。
- 设计测试用例是系统测试与组件测试的关键工作，主要是通过设计输入数据与预计的输出来测试系统。
- 测试用例设计的目的是建立一组测试用例集合，用尽可能少的测试代价有效的发现系统缺陷并证明系统能够满足需求。
- 设计测试用例的常用方法：
 - 划分测试与边界值分析；
 - 结构化测试（白盒测试）。





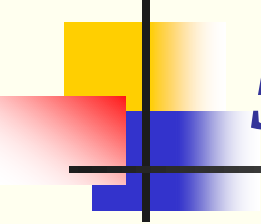
黑盒测试与白盒测试

- **黑盒测试**又叫做**功能测试**，测试者只关心系统的功能而不关心软件的实现。也就是说测试者不必了解有关系统的任何细节，只**把系统看成**是一个能够处理输入，产生输出的“**黑盒子**”，仅从功能的角度设计测试用例。
- **白盒测试**又叫做**结构测试**，是一种根据软件的结构知识导出测试用例的设计方法。测试者**把被测试组件看成**是一个打开的“**白盒子**”，组件的内部结构对测试者是透明的，通过对所用算法结构的分析设计测试用例。



5.10 等价划分测试

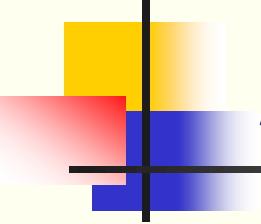
- 等价划分测试是测试用例设计的一种方法。设计测试用例时，可以按特征把数据输入域划分成若干等价类，等价类中的每个数据应该以同样的方式得到处理，因此对于揭露程序中的错误是等效的。这样，就可以选取少量有代表性的输入数据作为测试数据，以期用较小的代价暴露较多的程序错误。
- （看例题和练习题，要求会做类似习题）



5.11 结构化测试

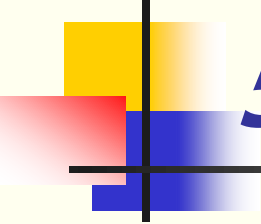
- 结构化测试是根据软件的结构知识导出测试用例的测试方法。
- 又叫做“**白盒测试法**”。
- 对组件中所用的算法结构的理解可以帮助我们找出更多的测试用例。





5.12 逻辑覆盖法

- 逻辑覆盖一系列测试过程的总称，这组测试会逐渐进行越来越完整的通路测试。
- 按照**不同的测试目标**，逻辑覆盖又可分为：**语句覆盖、判定覆盖、条件覆盖、判定 - 条件覆盖、条件组合覆盖及路径覆盖**。其中语句覆盖覆盖度最弱，路径覆盖最强！

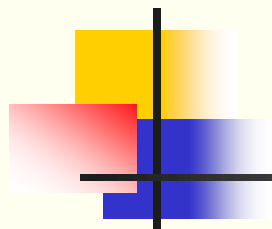


5.13 基本路径测试

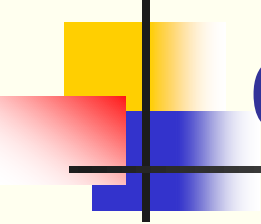
- 基本路径测试的原理：

“在程序控制流图的基础上，分析控制结构的环路复杂度，并用这个复杂度为指南定义执行路径的基本集合，从而导出基本可执行路径集合，设计出测试用例并保证**每个可执行语句至少执行一次，而且每个条件在执行时都将分别取真、假两种值。**”

- （掌握本讲课后作业与课上练习）

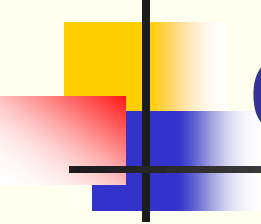


第六讲 软件维护



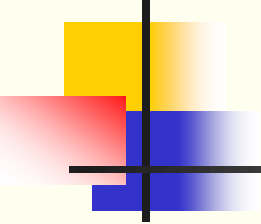
6.1 系统变更与进化

- 软件系统变更是不可避免的，软件系统开发的一个关键的问题就是如何实现和管理现存软件的变更；
- 软件系统随变更要求不断更新改进的过程就是系统进化过程。



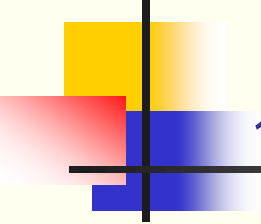
6.2 软件维护与维护成本

- 软件维护是指在软件交付使用之后，为了改正错误和满足新的需要而修改软件的过程。
- 维护通常包括四种类型：
 - **纠正性维护**——修补系统缺陷的维护，日常维护的主要工作。
 - **适应性维护**——使软件适应不同的操作环境（软硬件环境）的维护
 - **完善性维护**——增加或修改系统功能的维护
 - **预防性维护（再工程）**——为预防系统后期可能的失效而做的维护。



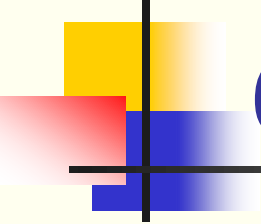
维护成本

- 维护成本一般会高于开发成本；
- 影响维护成本的有技术因素也有非技术因素。
- 开发阶段多些投入，有助于减少维护成本。
- 软件年龄越大，维护成本越高。



影响维护成本的主要因素

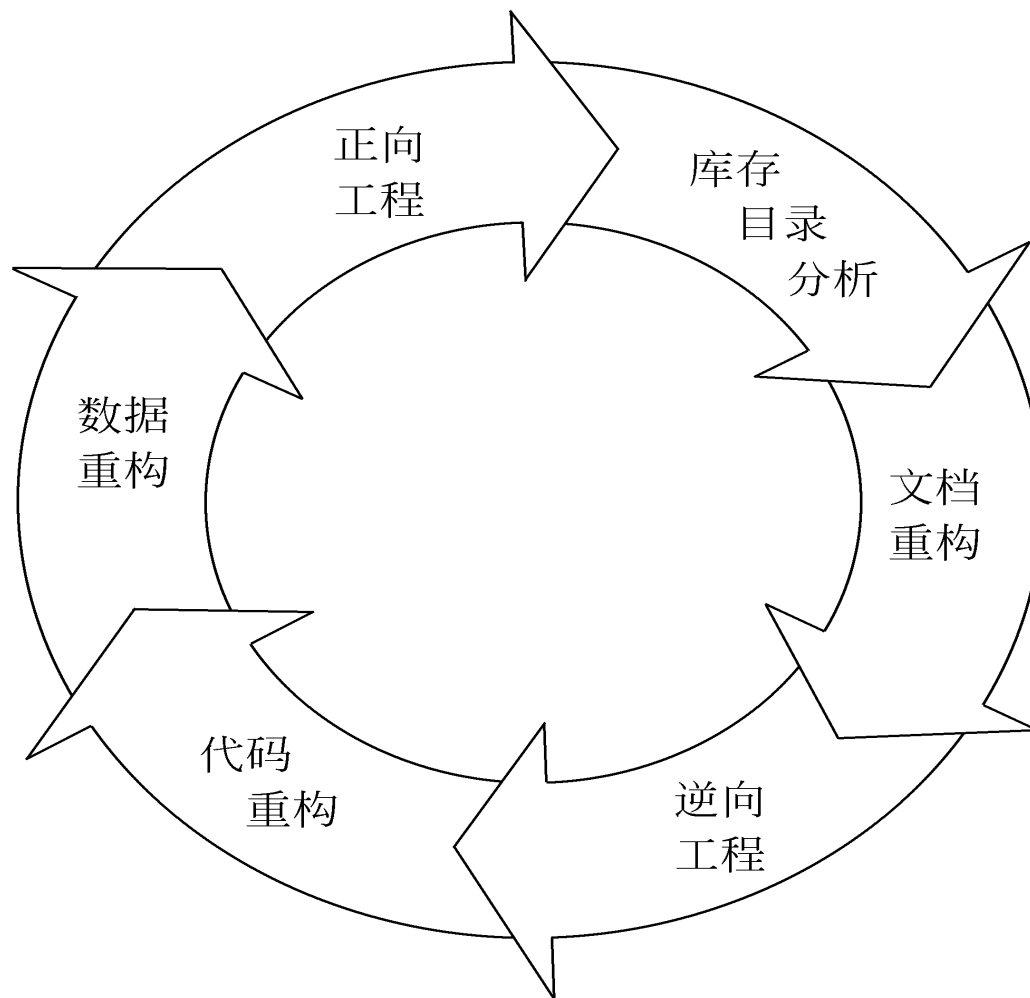
- 团队稳定性
- 合同责任
- 人员技术水平
- 程序年龄与结构

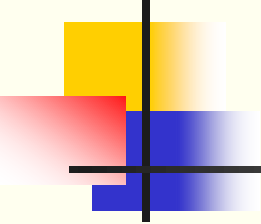


6.3 预防性维护/软件再工程

- 预防性维护, 或再工程, 是由Miller提出来的, 他的想法是“结构化翻新”。他把这种方法定义为: **“把今天的方法学应用到昨天的系统上, 以支持明天的需求。”**

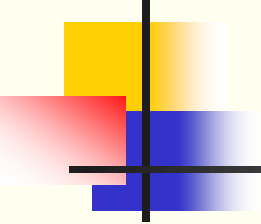
6.4 软件再工程过程模型



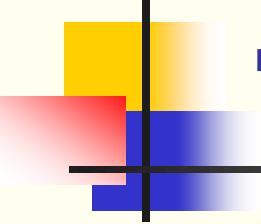


软件再工程过程

- 典型的软件再工程过程模型定义了6类活动。在某些情况下这些活动以线性顺序发生，但也并非总是这样，例如，为了理解某个程序的内部工作原理，可能在文档重构开始之前必须先进行逆向工程。
- 再工程范型是一个循环模型。这意味着作为该范型的组成部分的每个活动都可能被重复，而且对于任意一个特定的循环来说，过程可以在完成任意一个活动之后终止。

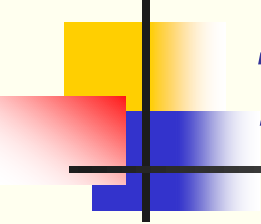


第七讲 软件项目管理



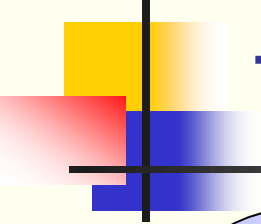
7.1 软件项目管理基础

- **软件项目管理**是为了使软件项目能够按照预定的成本、进度、质量顺利完成，而对成本、人员、进度、质量、风险等进行分析和管理的活动。
- 软件项目管理主要考虑如何保证**软件能够按时、按计划并满足用户需求规格**的交付，即如何用科学的管理手段保障软件项目的成功。



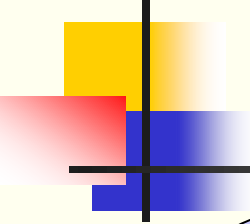
7.2 软件项目管理中的4 P's

- Pressman认为有效的软件项目管理集中在4个P上，即：
 - **人员 (People)** — “人的因素”是成功软件项目中最为重要的因素；
 - **产品 (Product)** — 产品的目标与范围，成本与开发约束是划分项目任务，制定项目进度的依据；
 - **过程 (Process)** — 软件过程提供了完成特定软件项目所需的框架活动和开发任务的集合；
 - **项目 (Project)** — 把软件置于有计划的、可控的项目之中，是保证其成功的唯一途径。



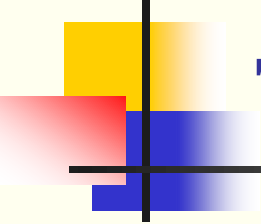
7.3 项目策划任务集

1. 确定项目范围;
2. 确定可行性;
3. 分析风险;
4. 确定所需的资源:
 - a. 确定需要的人力资源;
 - b. 确定可复用的软件资源;
 - c. 标识环境资源。



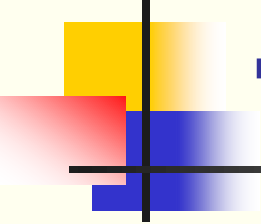
7.3 项目策划任务集

5. 估算成本和工作量：
 - a. 分解问题；
 - b. 使用规模、功能点、过程任务或用例等方法进行两种以上的估算；
 - c. 调和不同的估算。
6. 制定项目进度计划：
 - a. 建立一组有意义的任务集合；
 - b. 定义任务网络；
 - c. 使用进度计划工具指定时间表；
 - d. 定义进度跟踪机制。



7.4 软件项目的成本构成

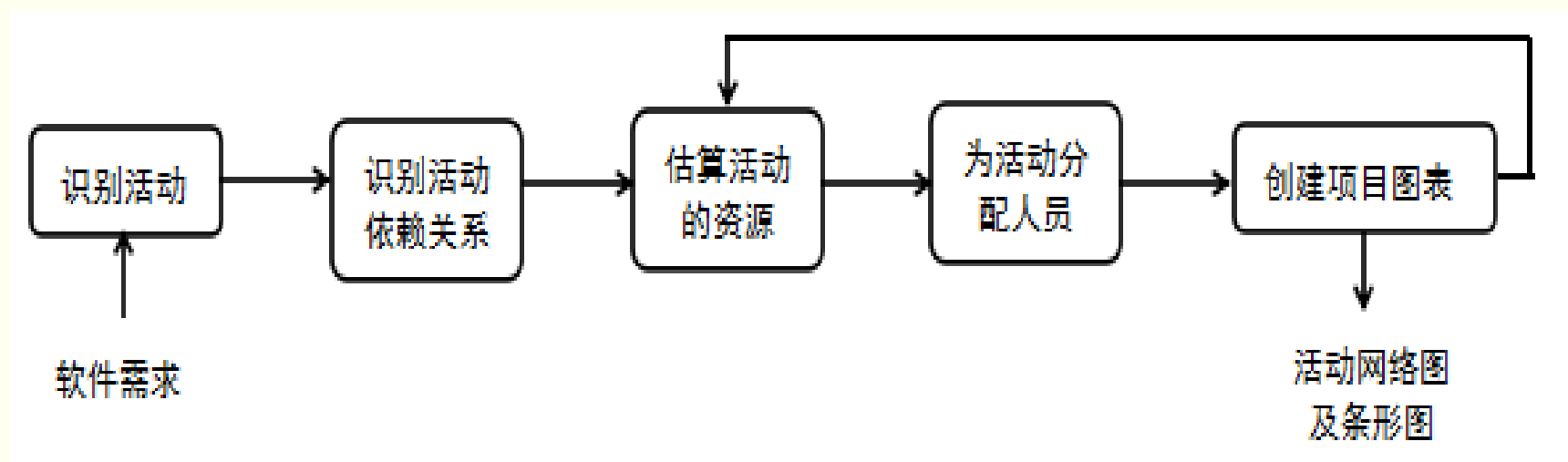
- 硬件和软件成本
- 差旅费和培训费用
- 工作成本 (the dominant factor in most projects)
 - 项目开发人员的薪水;
 - 社会保障和员工福利。
- 经常性的管理费用
 - 办公场所、供暖和照明费用;
 - 网络和通信费用;
 - 图书馆、员工餐厅等方便设施的费用。

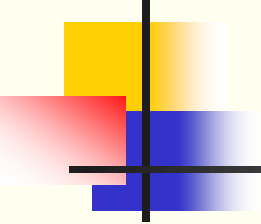


7.5 项目进度安排（调度）

- 这个管理活动中，管理者需要把项目分解成若干个任务，并估算每个任务完成所需要的时间与资源，然后按照一定的顺序把这些任务组织起来。
- 应协调并行的任务，充分利用人力资源。
- 要减少任务间的依赖，明确关键任务，保证按进度交付。

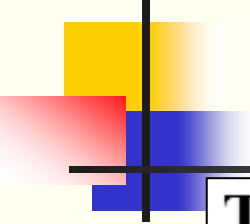
项目调度过程





活动网络图与甘特图

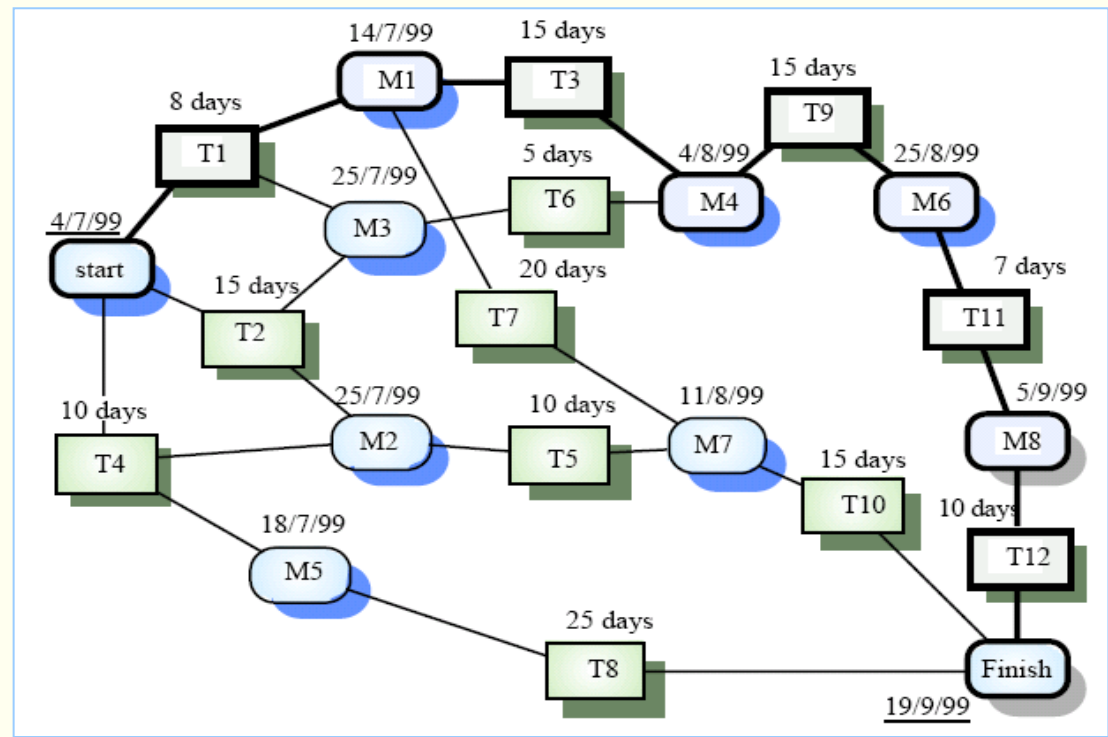
- 图形化的工具对说明项目进度是十分有用的。
- **活动网络图**表示构成一个项目的不同活动之间的依赖关系以及由开始到结束的**关键活动路径**。

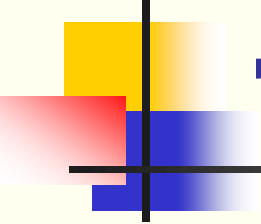


例：活动网络图的画法

Task	Duration (days)	Dependencies
T1	8	
T2	15	
T3	15	T1 (M1)
T4	10	
T5	10	T2, T4 (M2)
T6	5	T1, T2 (M3)
T7	20	T1 (M1)
T8	25	T4 (M5)
T9	15	T3, T6 (M4)
T10	15	T5, T7 (M7)
T11	7	T9 (M6)
T12	10	T11 (M8)

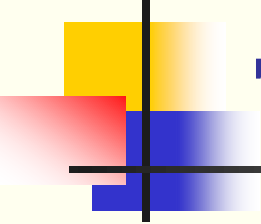
Task	Duration (days)	Dependencies
T1	8	
T2	15	
T3	15	T1 (M1)
T4	10	
T5	10	T2, T4 (M2)
T6	5	T1, T2 (M3)
T7	20	T1 (M1)
T8	25	T4 (M5)
T9	15	T3, T6 (M4)
T10	15	T5, T7 (M7)
T11	7	T9 (M6)
T12	10	T11 (M8)





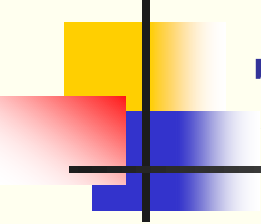
7.6 质量管理

- CMM 对质量的定义是：① 一个系统、组件或过程符合特定需求的程度；② 一个系统、组件或过程符合客户或用户的要求或期望的程度。
- **可以这样理解软件质量：** 软件质量是许多**质量属性**的综合体现，各种质量属性反映了软件质量的方方面面。人们通过改善软件的各种质量属性，从而提高软件的整体质量。
- **软件质量要素**是指对软件整体质量影响最大的质量属性或者客户最关心的、能成为卖点的质量属性



7.6 质量管理

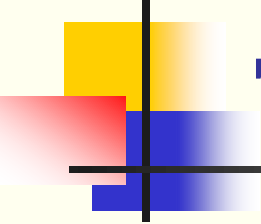
- 软件质量属性可分为两大类：
 - 功能性质量属性：正确性，健壮性，可靠性。
 - 非功能性质量属性：性能，易用性，安全性，可扩展性，兼容性，可移植性。
- 可以用来评价软件质量的6个质量要素包括：
 - 功能性、可靠性、易用性、效率、可维护性、可移植性



7.7 配置管理

配置管理规程规定了如何记录和处理所提议的变更，如何使系统变更与系统组件相关联，以及如何识别系统不同版本的方法。

配置管理的目的在于控制由变更带来的成本和人力消耗。



7.7 配置管理

- **软件配置 (SC) :**
 - 软件过程中产生的所有信息总称为软件配置。
 - 软件过程的输出信息主要有三类：（1）计算机程序；（2）描述计算机程序的文档；（3）数据。
- **软件配置项 (SCI)** 是在软件过程中至于配置管理之下的输出信息，通常表现为一组正式文档。
- **基线 (base lines) :** 已经通过正式评审和批准的规格说明或产品，他可以作为进一步开发的基础，并且只有通过正式的变更控制规程才能修改它。